



University of Antwerp

Faculty of Applied  
Engineering

# **4-Computerarchitectuur en besturingssystemen 2 overzicht**

Laatste aanpassing: 10/03/2026

E.Paillet

- 15 practica van 4u (vanaf lesweek 3)
  - 9 sessies computerarchitectuur:
    - 3 sessies core (assembler)
    - 3 sessies CPU hardware (VHDL)
    - 3 sessies periferie (C)
  - 5 sessies Operating Systemen
  - 1 sessie evaluatie
    - per onderdeel: test van 1 uur

# Beoordeling

- Portfolio voor core, periferie en OS
- Test tijdens laatste sessie voor alle vier de modules
- Eindscore:
  - core: 20%
    - portfolio (20%) + test (80%)
  - CPU hardware: 20%
    - test (100%)
  - periferie: 20%
    - portfolio (20%) + test (80%)
  - OS: 40%
    - portfolio (20%) + test (80%)

# Beoordeling

- Minimum 80% aanwezigheid (12 sessies), anders max 7/20!
- Minimum 30% per onderdeel, anders max 8/20!
- Modules waarvoor geslaagd: punten blijven behouden in tweede zit
- Geen deelloverdrachten naar het volgend academiejaar
- Evaluatiecriteria portfolio:
  - gestructureerd
  - bevat de verslagen van de opdrachten
- Test tijdens laatste sessie:
  - oefeningen gelijkaardig aan die van tijdens de practica



University of Antwerp  
Faculty of Applied  
Engineering

# **4-Computerarchitectuur en besturingssystemen: Microcontroller periferie**





**University of Antwerp**  
Faculty of Applied  
Engineering

# **Inleiding**



# Doelstellingen

- Doel van het labo:
  - Eigenschappen specifiek aan  $\mu$ C kennen
  - Periferie van een  $\mu$ C kunnen gebruiken om te interfacen met externe componenten
  - Kunnen werken met technische documentatie zoals processor manuals en datasheets



# Vereiste voorkennis

- C / C++
- Basiskennis analoge en digitale elektronica (passieve kennis)





- Geen templates (STL)
- Geen klassen, wel structs
- Geen stream IO library → stdio.h gebruiken: printf, scanf, ... → zie ook 'printf en scanf' info op blackboard.
- Include files: niet <cstdio> wel <stdio.h>, niet <cmath> wel <math.h>,...
- Standaard C library → LIBC manual



# Documentatie

- Labotekst
- Powerpoint presentatie
- 'Processor manual': XMEGACManual.pdf
- 'LIBC manual': libc-user-manual.pdf
- Documentatie, software, PPT: zie blackboard



- 20%: Portfolio:

- Correcte en volledige uitvoering van gevraagde opdrachten in cursustekst.
- Gedocumenteerde code (inline commentaar)
- Code moet modulair en goed gestructureerd zijn

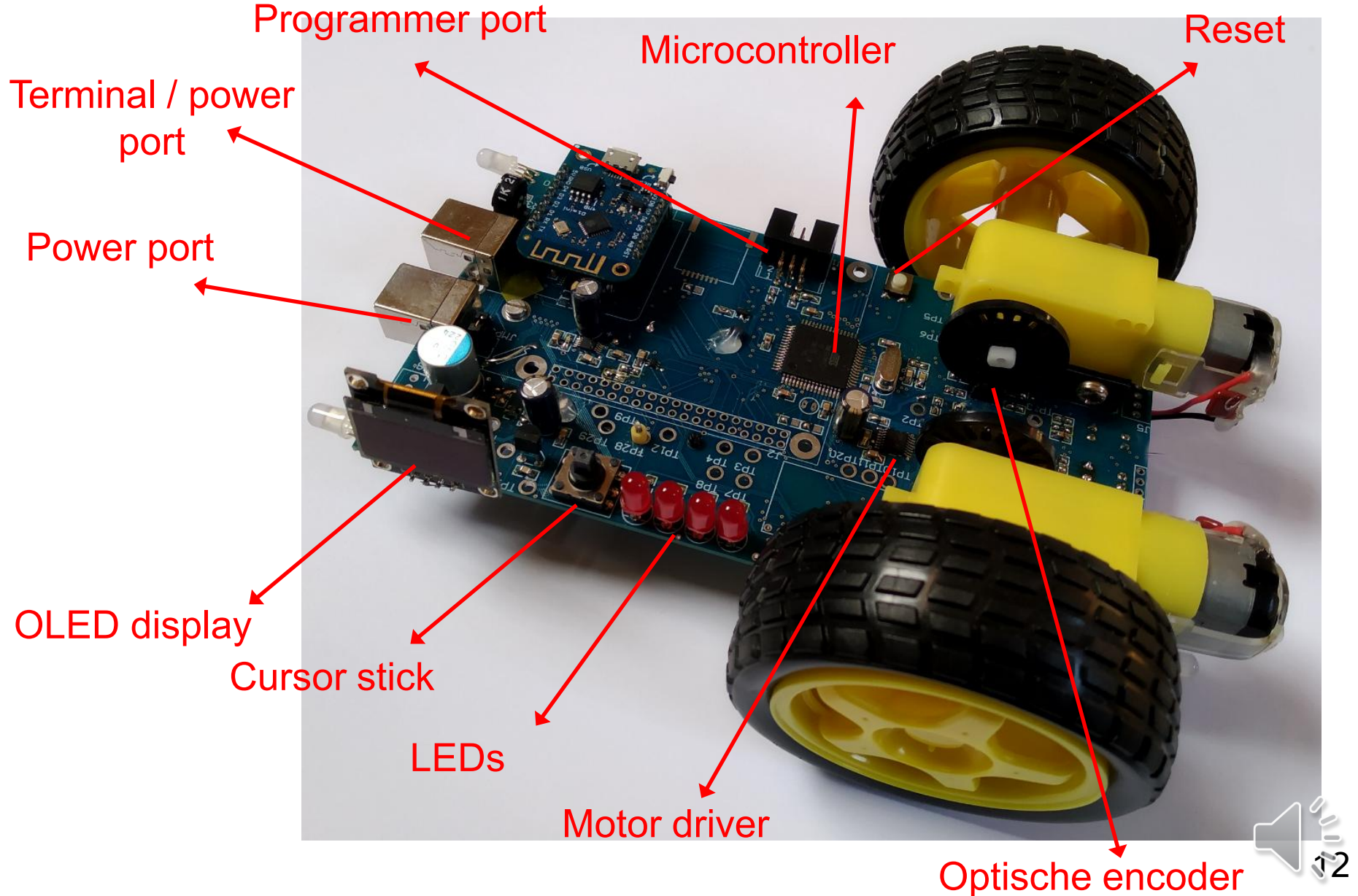
In te dienen: Volledig code project + baud rate tool, metingen

- 80%: Test:

- Oefening(en) analoog met uitgevoerde opdracht
- Op papier, zonder robot HW
- Open boek echter zonder internet connectie



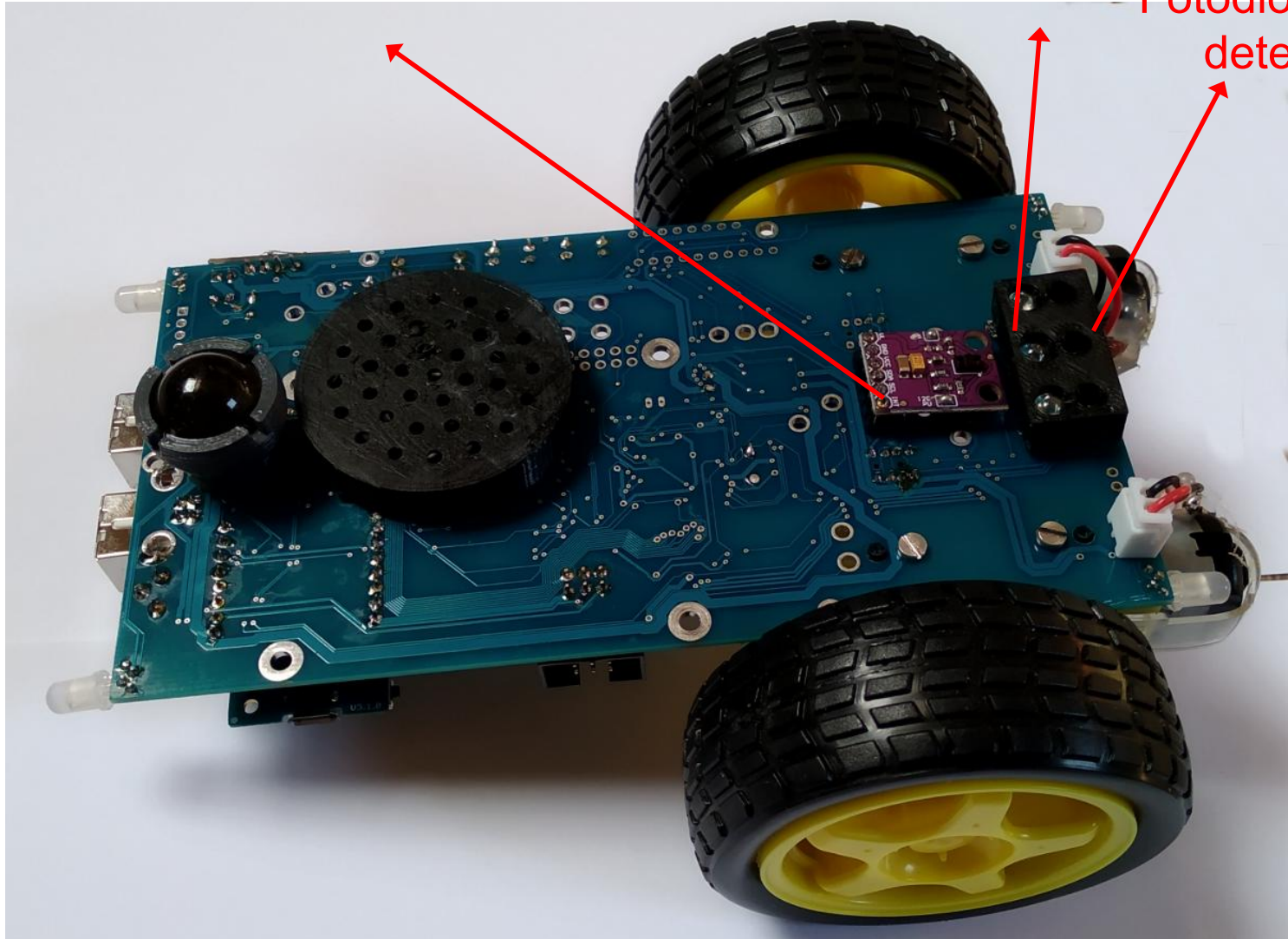
# Praktische opdracht: Linebot



Kleursensor

LEDs lijn detector

Fotodiodes lijn  
detector





University of Antwerp  
Faculty of Applied  
Engineering

# Atmel XMEGA $\mu$ C familie



# Microcontrollers: definitie

- Microprocessor + periferie geïntegreerd
- Periferie
  - Oscillators
  - Geheugen
  - Communicatie (SPI, UART, USB, Ethernet, ...)
  - Timers
  - ADC's, DAC's
- Meestal zeer energiezuinig
- Meestal lage kostprijs

# Atmel XMEGA familie

- Moderne 8 bit controllerreeks
- Relatief eenvoudige structuur t.o.v. bv CORTEX Mx microcontrollers
- Breed gamma, sub 1 EUR tot ca 15 EUR
- Tot 32 MIPS
- Harvard architectuur (gescheiden programma, data geheugen)
- Pincompatibele reeksen
- Verbruik: max 30mW (PC processor: 100 W!)
- Toepassingen: battery management, motor control, sensor en actuator interfacing, automotive, domotica, ...





# Typeconventies

- ATXMEGA128A4U

Programmageheugen lengte (in kByte)

384

256

192

128

64

32

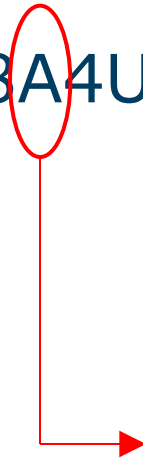
16

(\*) [http://www.atmel.com/products/microcontrollers/avr/avr\\_xmega.aspx](http://www.atmel.com/products/microcontrollers/avr/avr_xmega.aspx)



# Typeconventies

- ATXMEGA128A4U



Periferie rijkheid

E: minste periferie modules

D:

C:



B:

A: meeste periferie modules

# Typeconventies

- ATXMEGA128A4U



Aantal chip pinnen

4: 44

3: 64

1: 100

# Typeconventies

- ATXMEGA128A4U



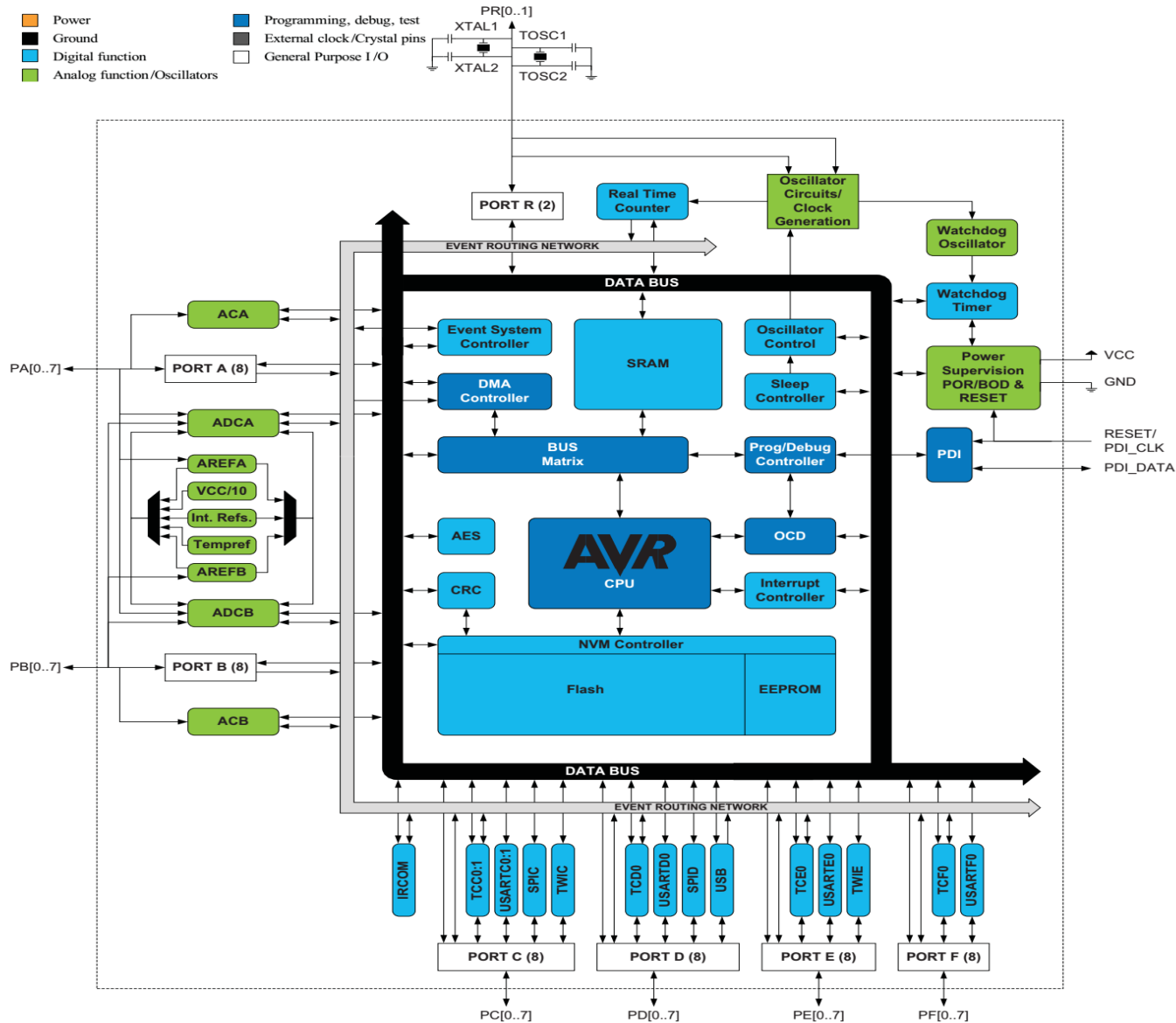
U: USB interface aanwezig



# ATXMEGA384C3U

## Zie blokschema 'processor manual' p4

Figure 2-1. Atmel AVR XMEGA C Block Diagram



# ATXMEGA384C3

- 384 + 8kB FLASH programma geheugen
- 32kB SRAM data geheugen
- 4kB EEPROM geheugen
- Max 32 MHz CLK
- 64 pins package
- Gebruikte periferie: GPIO, USART, ADC, TC, TWI, PDI, interrupt system
- Andere periferie: 'processor manual'



# Pin multiplexing

- Slechts 64 pinnen, echter veel meer functies  
→ pinnen hebben meerdere functies.
- Zie tabellen labotekst appendix A

| PORT C | PIN # | INTERRUPT      | TCC0 <sup>(1)(2)</sup> | AWEXC                      | TCC1 | USARTC0 <sup>(3)</sup> | SPIC <sup>(4)</sup>    | TWIC | CLOCKOUT <sup>(5)</sup> | EVENTOUT <sup>(6)</sup> |
|--------|-------|----------------|------------------------|----------------------------|------|------------------------|------------------------|------|-------------------------|-------------------------|
| PC0    | 16    | SYNC           | OC0A                   | $\overline{\text{OC0ALS}}$ |      |                        |                        | SDA  |                         |                         |
| PC1    | 17    | SYNC           | OC0B                   | OC0AHS                     |      | XCK0                   |                        | SCL  |                         |                         |
| PC2    | 18    | SYNC/<br>ASYNC | OC0C                   | $\overline{\text{OC0BLS}}$ |      | RXD0                   |                        |      |                         |                         |
| PC3    | 19    | SYNC           | OC0D                   | OC0BHS                     |      | TXD0                   |                        |      |                         |                         |
| PC4    | 20    | SYNC           |                        | $\overline{\text{OC0CLS}}$ | OC1A |                        | $\overline{\text{SS}}$ |      |                         |                         |
| PC5    | 21    | SYNC           |                        | OC0CHS                     | OC1B |                        | MOSI                   |      |                         |                         |
| PC6    | 22    | SYNC           |                        | $\overline{\text{OC0DLS}}$ |      |                        | MISO                   |      | RTCOUT                  |                         |
| PC7    | 23    | SYNC           |                        | OC0DHS                     |      |                        | SCK                    |      | clk <sub>PER</sub>      | EVOUT                   |
| GND    | 24    |                |                        |                            |      |                        |                        |      |                         |                         |
| VCC    | 25    |                |                        |                            |      |                        |                        |      |                         |                         |





University of Antwerp  
Faculty of Applied  
Engineering

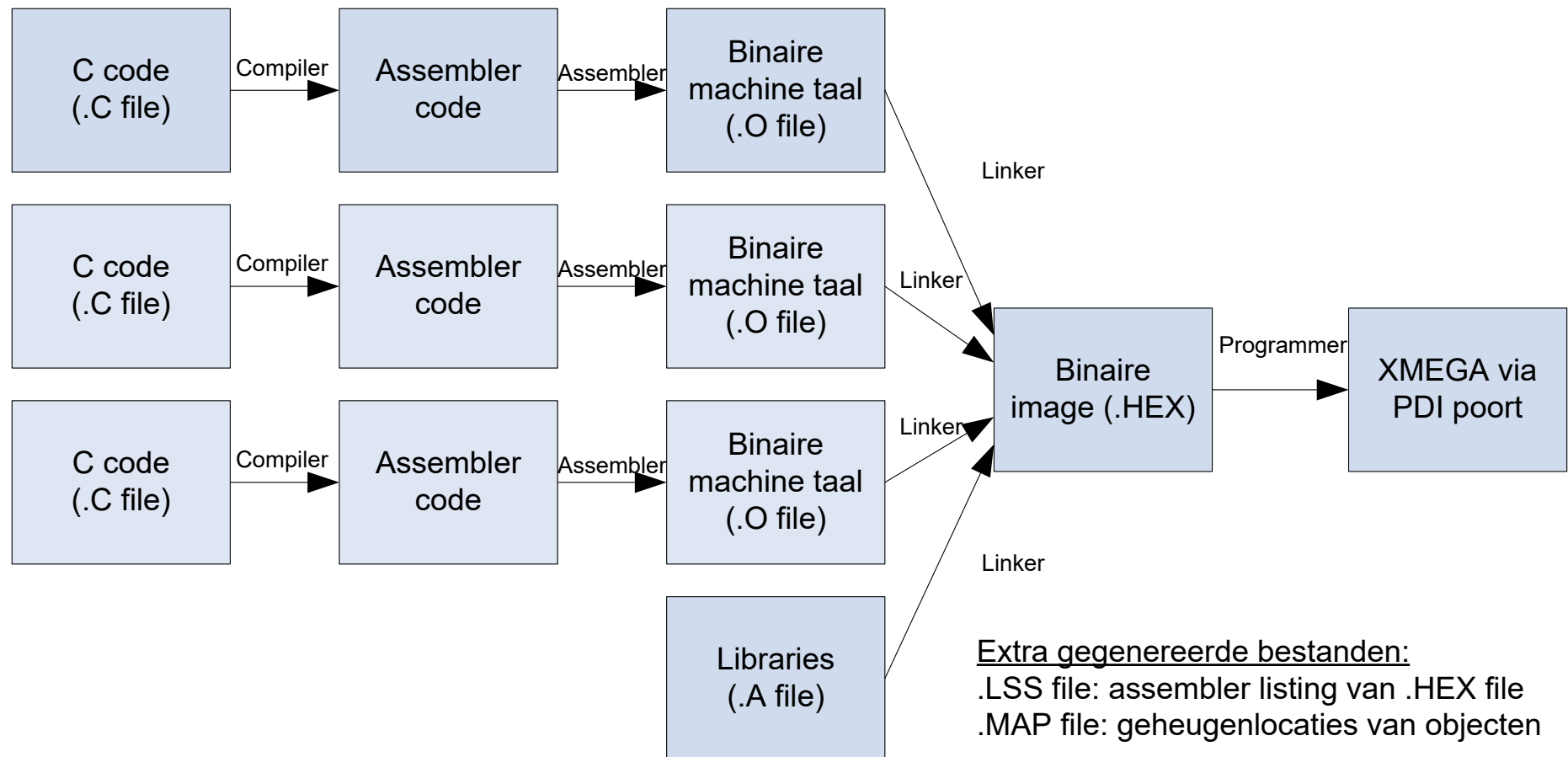
# Geheugens (FLASH, EEPROM, RAM)



# FLASH prog. geheugen

- Gebruikt om programmacode (binaire machinetaal) op te slaan
- Niet volatiel
- 384kB = +/- 60 000 regels C code
- NIET toegankelijk via datapointers (Harvard architectuur!)
- Programmeerbaar:
  - Via PDI interface
  - Door eigen code (bv bootloader)

# FLASH prog. geheugen



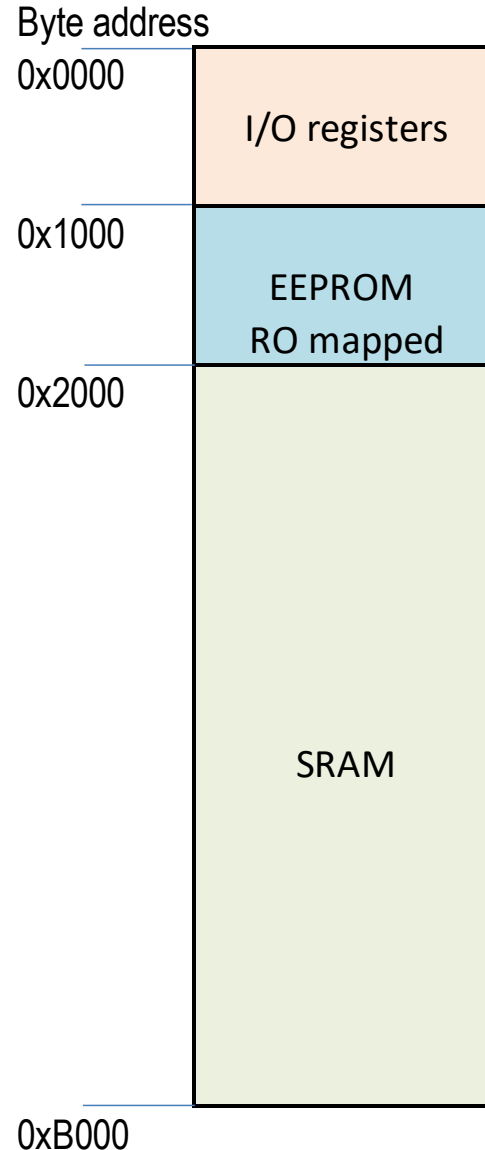
- C code begint uitvoer via **int main (void)** functie



# Data geheugen

## ATXMEGA384C3 General data memory map

- I/O reg: controle periferie
- EEPROM: read only memory mapped
- Interne SRAM: stockage ruimte voor variabelen



# I/O registers

- Bieden toegang tot alle periferie: controle, status, data
- Toegankelijk in C vanuit `#include <avr/io.h>`
- I/O registers worden weergegeven als structs
  - bv: `ADCA.CH0.CTRL`: Analog to Digital converter A: Channel 0: Control register
  - bv: `PORTA.IN`: General purpose I/O port A: Input register
- Zelfde naamgeving als processor manual



# Datageheugen: datatypes

- char : 8 bit
- int = short : 16 bit
- long: 32 bit
- float = double : 32 bit



- FLASH: programmeerbaar per bank
- EEPROM: programmeerbaar per byte
- Nuttig om variabelen op te slaan die niet verloren mogen gaan na uitschakelen spanning
- Bv: calibratiedata, paswoorden, foutcodes, ...



University of Antwerp  
| Faculty of Applied  
Engineering

# Processor klok

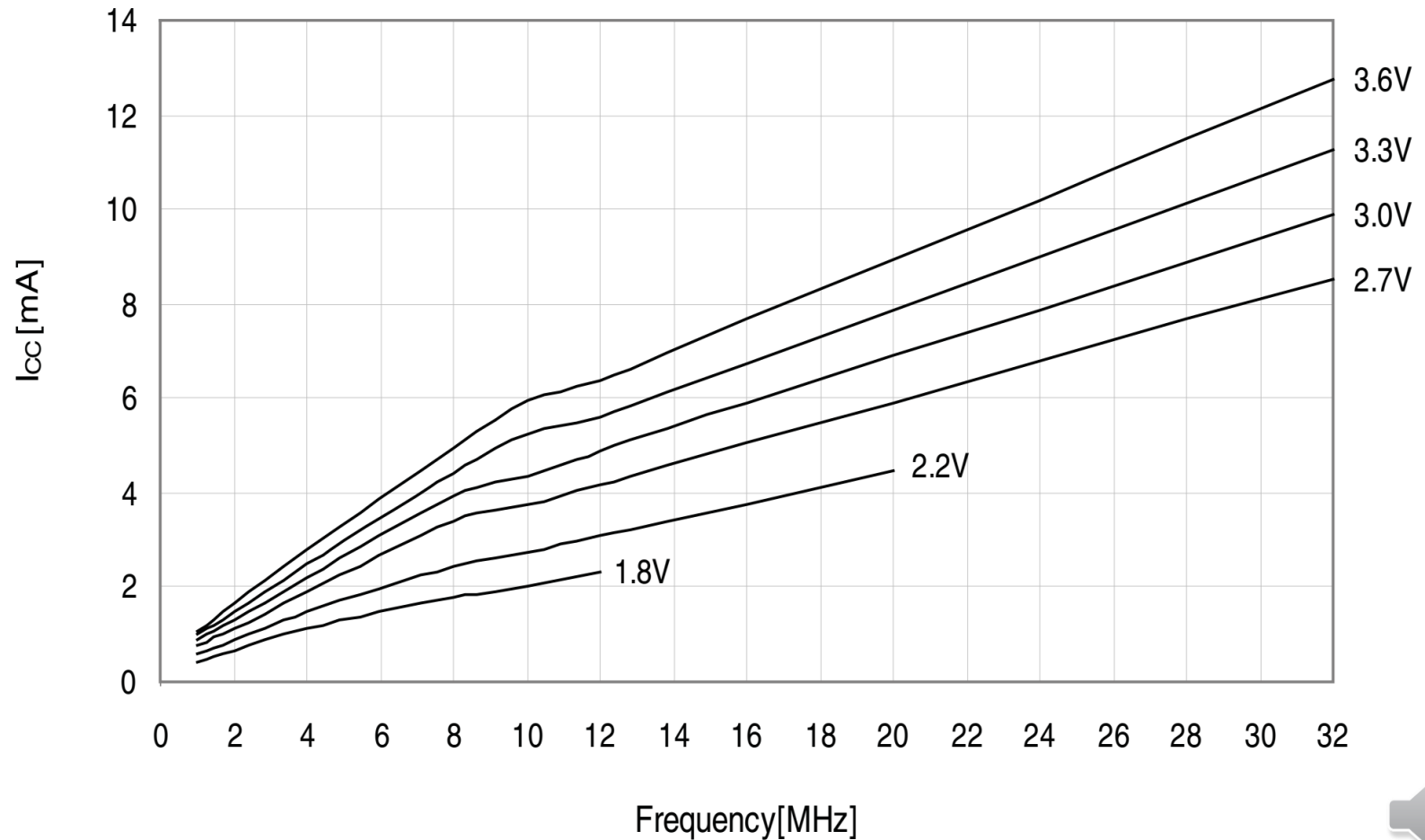
- $\mu$ C: synchroon digitaal systeem  $\rightarrow$  klok nodig
- Mogelijke bronnen:
  - Extern signaal
  - Extern kristal, interne oscillator
  - Interne RC oscillator
- Verschillende subsystemen hebben verschillende frequenties nodig  $\rightarrow$  interne opdeling / vermenigvuldiging
- Interne instelbare PLL  $\rightarrow$  klokfrequentie vermenigvuldiging is mogelijk.

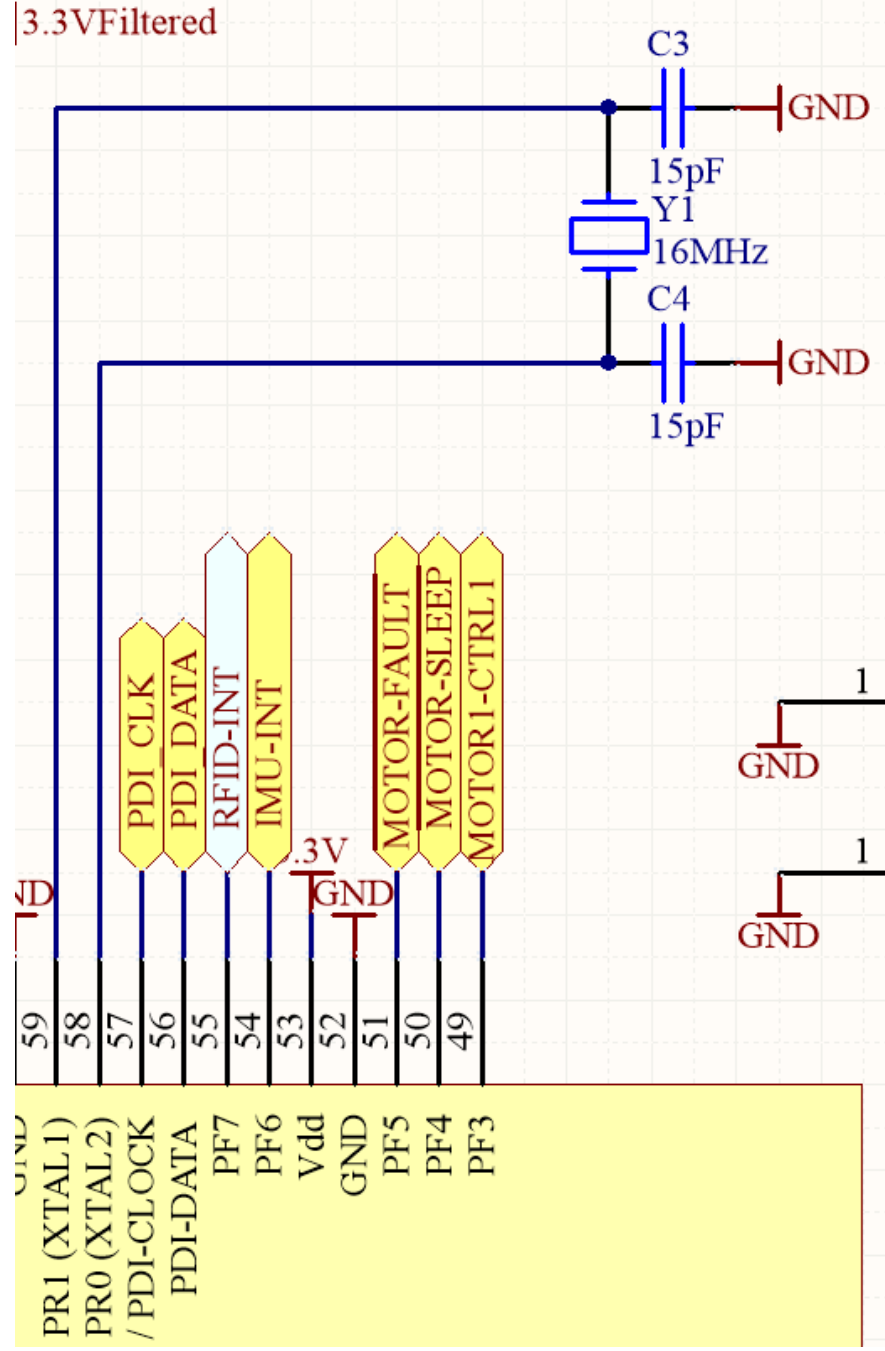


# Klok: invloed op energieverbruik

**Figure 36-2. Active Supply Current vs. Frequency**

$f_{\text{SYS}} = 1 - 32\text{MHz}$  external clock,  $T = 25^\circ\text{C}$







University of Antwerp  
Faculty of Applied  
Engineering

# I/O ports (GPIO) Manual p 119 e.v.

- Meeste pinnen kunnen als GPIO gebruikt worden
- GPIO: elke pin kan als dig. input / output gebruikt worden
- Interrupt mogelijk op stijgende flank, dalende flank, laag niveau → zie later
- Elke pin: elektrische configuratie is instelbaar
- Elke pin: optioneel inverteerbare logica
- Gegroepeerd in 'ports' A,B,C,D,E,F,R
- Elke port: max 8 pinnen (1 bit per pin)

# IO registers: DIR registers

x: A,B,C,D,E,R

Bepaalt de data transfer richting van de poort pinnen (output of input)

**PORTx.DIR**: directie register. 1 bit per pin.  
0=input, 1=output

Bv **PORTB.DIR**: 0b00110011

PB0: uitgang

PB4:uitgang

PB1: uitgang

PB5:uitgang

PB2: ingang

PB6:ingang

PB3: ingang

PB7:ingang

# IO registers: OUT registers

x: A,B,C,D,E,F,R

Bepaalt in output mode het uitgangsniveau van een poort pin

Alleen invloed indien DIR van de pin op 1 staat.

**PORTx.OUT**: output register. 1 bit per pin.

0=0V, 1=3,3V

# IO registers: IN registers

x: A,B,C,D,E,F,R

Leest de huidige toestand van de poort pinnen in

**PORTx.IN**: input register. 1 bit per pin. 0=0V,  
1=3,3V

# IO registers: PINnCTRL registers

Configuratieregister, per poort pin één register:

x: A,B,C,D,E,F,R

n:0-7

## PORTx.PINnCTRL:

| Bit           | 7   | 6     | 5        | 4   | 3   | 2        | 1   | 0   |
|---------------|-----|-------|----------|-----|-----|----------|-----|-----|
|               | –   | INVEN | OPC[2:0] |     |     | ISC[2:0] |     |     |
| Read/Write    | R/W | R/W   | R/W      | R/W | R/W | R/W      | R/W | R/W |
| Initial Value | 0   | 0     | 0        | 0   | 0   | 0        | 0   | 0   |

bit 0-2: ISC; input sense config: zie interrupts

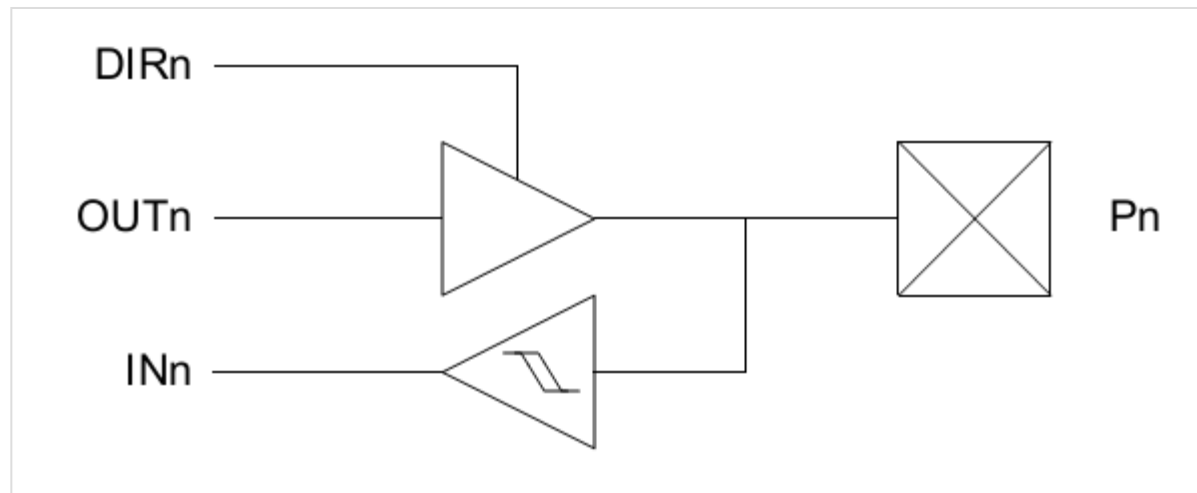
bit 3-5: OPC; output config (zie volgende slides)

bit 6: INVEN; indien 1 wordt de logica van inputs en outputs geïnverteerd

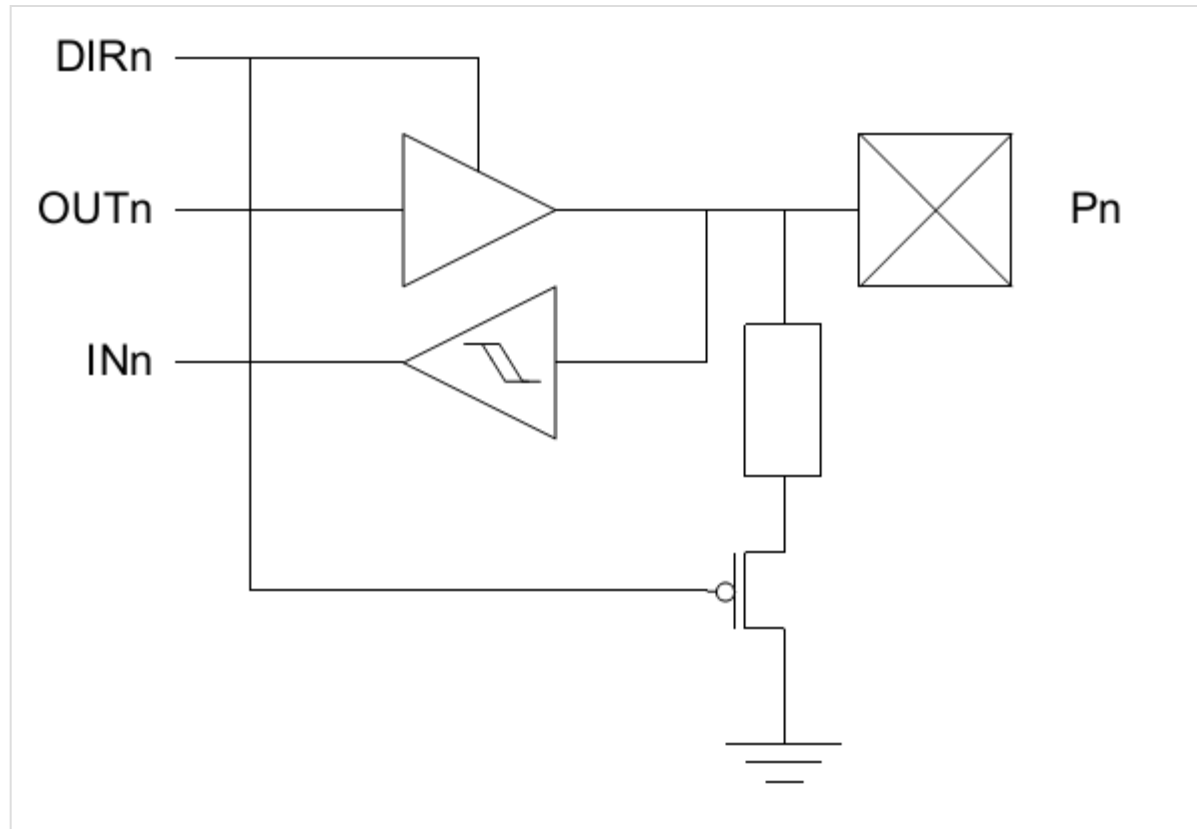




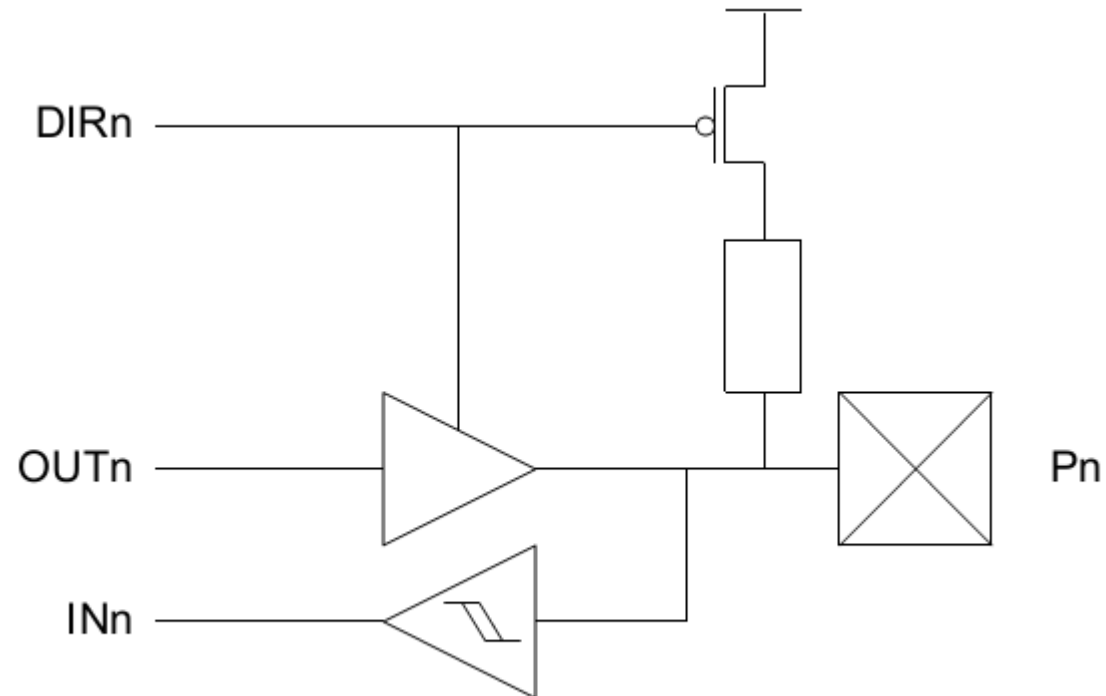
# Elektrische config : totem pole (000)



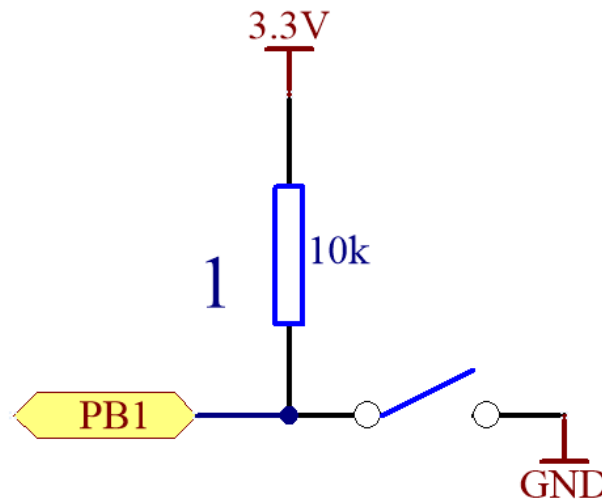
# Elektrische config : totem pole + pull down (010)



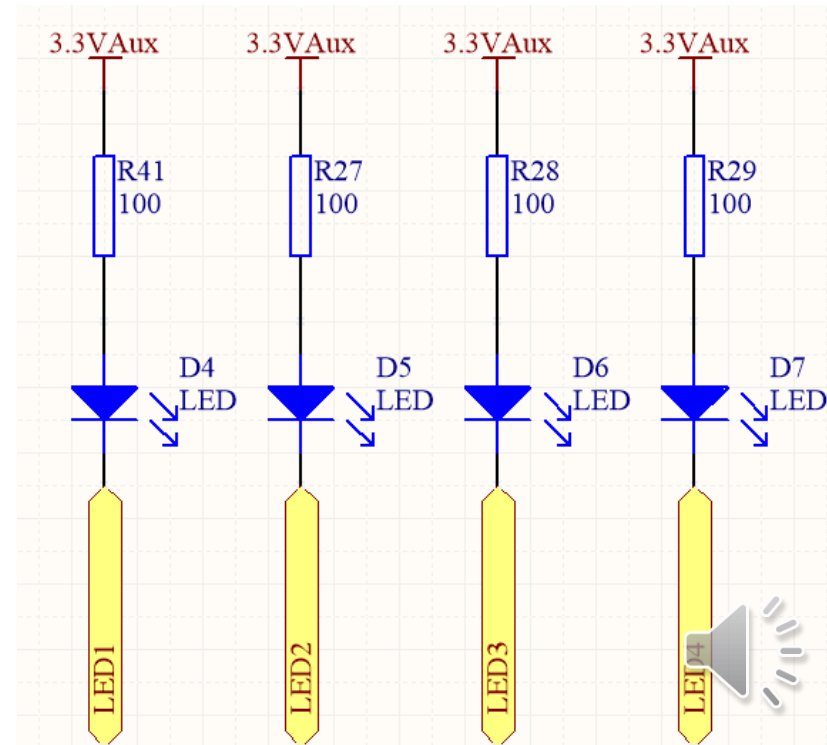
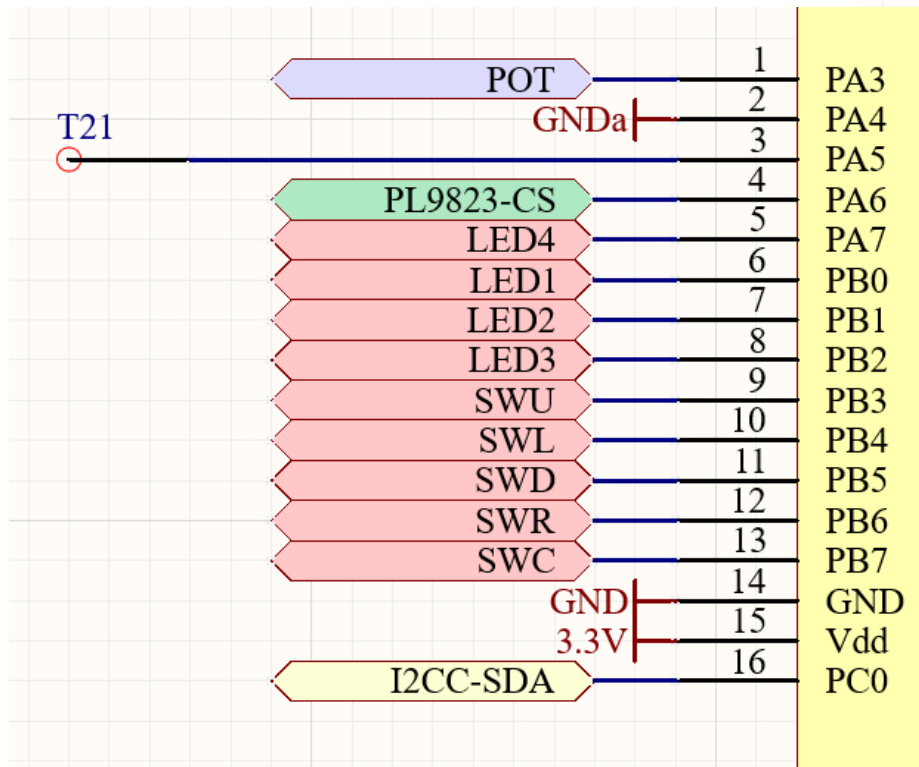
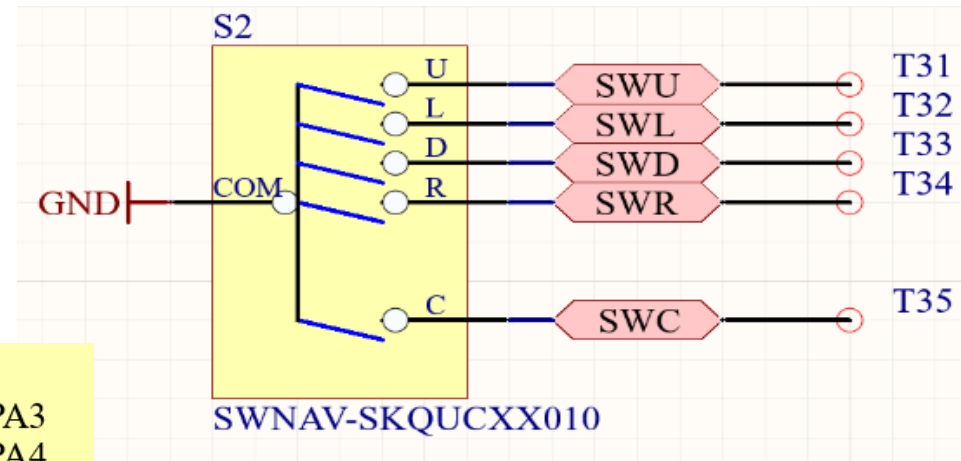
# Elektrische config : totem pole + pull up (011)



# Waarom pull-up / pull-down weerstanden?



# Hardware: GPIO (LEDs, cursor stick)





University of Antwerp  
| Faculty of Applied  
Engineering

# Debugging



# Debugging

- Breakpoint / stap voor stap debugging
  - **Voordeel:** programma stap voor stap doorlopen tot fout optreedt. Elke variabele kan gemonitord worden.
  - **Nadeel:** programma wordt stilgelegd → niet realtime. Vaak problemen met compiler code optimalisatie
- Seriële terminal (USART)
  - **Voordeel:** programma blijft lopen → debugging van realtime problemen
  - **Nadeel:** weinig flexibel. Elke aanpassing: hercompileren, code terug uploaden





University of Antwerp  
Faculty of Applied  
Engineering

# Operators, notaties in C Bit operaties



# Belangrijke operators:

- bitsgewijs:

|     |                     |
|-----|---------------------|
| &   | AND                 |
|     | OR                  |
| ^   | XOR                 |
| ~   | NOT                 |
| <<n | SHIFT<br>LEFT n     |
| >>n | SHIFT<br>RIGHT<br>n |



# Volgorde van bewerkingen:

- Eerst naar laatst:

\* / %

+ -

<< >>

< <= > >=

== !=

&

^

|

&&

||

= += -= \*= /= , ...

# Bitsgewijze set

We willen bit 1 en 7 in var x setten

| b     |    | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|-------|----|---|---|---|---|---|---|---|---|
| x :   | 0b | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 1 |
| x = x | 0b | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| x :   | 0b | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 1 |



# Bitsgewijze reset

We willen bit 1 en 7 in var x resetten

| b       |    | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---------|----|---|---|---|---|---|---|---|---|
| x :     | 0b | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 1 |
| x = x & | 0b | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 |
| x :     | 0b | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 1 |



# Bitsgewijze toggle

We willen bit 3,4,5 in var x togglen

| b       |    | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---------|----|---|---|---|---|---|---|---|---|
| x :     | 0b | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 1 |
| x = x ^ | 0b | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 0 |
| x :     | 0b | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 |



# Bitsgewijze selectie

We willen alleen de toestand van bit 6 te weten komen

| b       |    | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---------|----|---|---|---|---|---|---|---|---|
| x :     | 0b | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 1 |
| x = x & | 0b | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| x :     | 0b | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |

```
if (x)
{ ... }
else
{ ... }
```



- Decimaal: `char a=123;`
- Hexadecimaal `char a=0x7B;`
- Binair: `char a=0b01111011;`
- ASCII: `char a='{';`





University of Antwerp  
Faculty of Applied  
Engineering

# **TC (Timer / Counter)** **Manual p142 e.v.**



- TC: 16 bit teller, geklokt door de periferieklok via een instelbare prescaler (/1 tot /1024)
- 1 stap omhoog per klokpuls (na prescaler)
- Teller: Write, Read access (TCx.CNT)
- Loopt tot TCx.PER
- Overflow interrupt mogelijk
- Verscheidene 'Waveform Generation Modes'
- Capture modes: timen van externe signalen (wordt niet gezien in het labo)
- ATXMEGA384C3: 5 TC's (TCC0, TCC1, TCD0, TCE0, TCF0)

# Waveform Generation Modes (WGM)

- Automatisch (hardwarematig) genereren van golfvormen / interrupts
- Elke TC: 2 of 4 CC (Capture/Compare) registers: `TCx.CCA`, `TCx.CCB`, `TCx.CCC`, `TCx.CCD`
- Bij een CC kanaal hoort een 'Output Compare' pin. Mapping: zie cursus appendix A

# CC – OC verband

TCC0.CCA

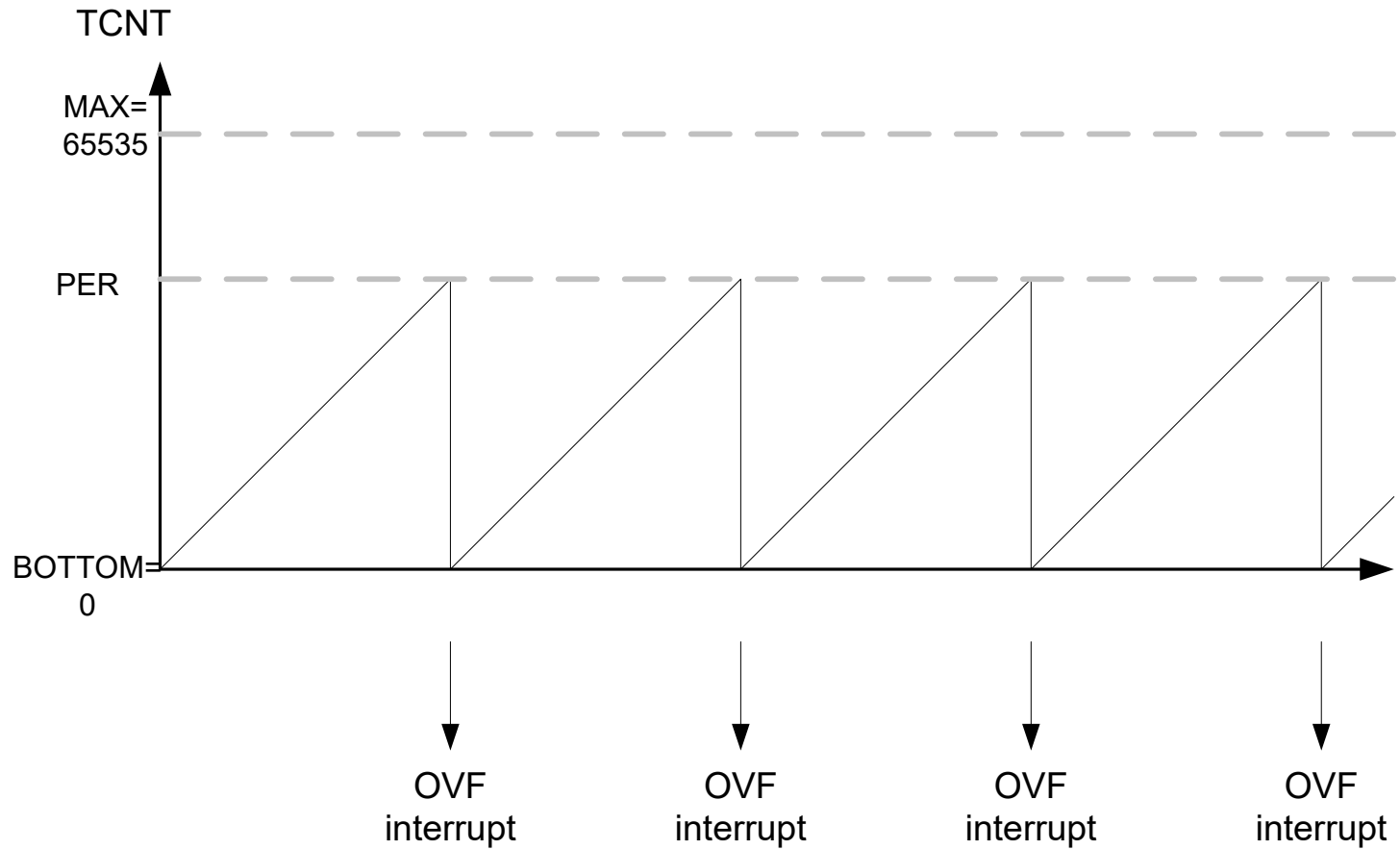
TCC0.CCB

TCC0.CCC

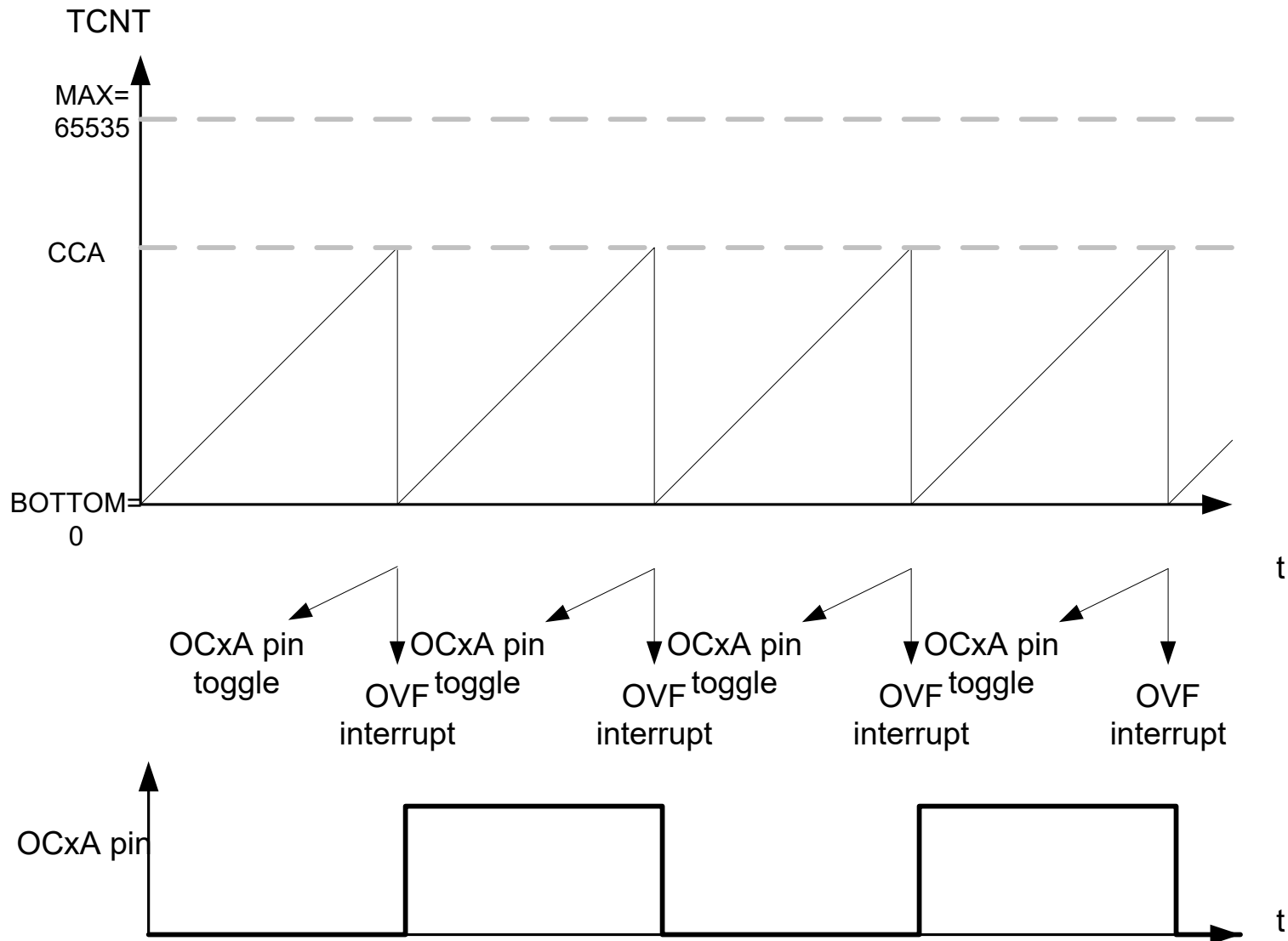
TCC0.CCD

| PORT C | PIN # | INTERRUPT      | TCC0 <sup>(1)(2)</sup> | AWEXC                      | TCC1 |  |
|--------|-------|----------------|------------------------|----------------------------|------|--|
| PC0    | 16    | SYNC           | OC0A                   | $\overline{\text{OC0ALS}}$ |      |  |
| PC1    | 17    | SYNC           | OC0B                   | OC0AHS                     |      |  |
| PC2    | 18    | SYNC/<br>ASYNC | OC0C                   | $\overline{\text{OC0BLS}}$ |      |  |
| PC3    | 19    | SYNC           | OC0D                   | OC0BHS                     |      |  |
| PC4    | 20    | SYNC           |                        | $\overline{\text{OC0CLS}}$ | OC1A |  |
| PC5    | 21    | SYNC           |                        | OC0CHS                     | OC1B |  |
| PC6    | 22    | SYNC           |                        | $\overline{\text{OC0DLS}}$ |      |  |
| PC7    | 23    | SYNC           |                        | OC0DHS                     |      |  |
| GND    | 24    |                |                        |                            |      |  |
| VCC    | 25    |                |                        |                            |      |  |

# Normal WGM

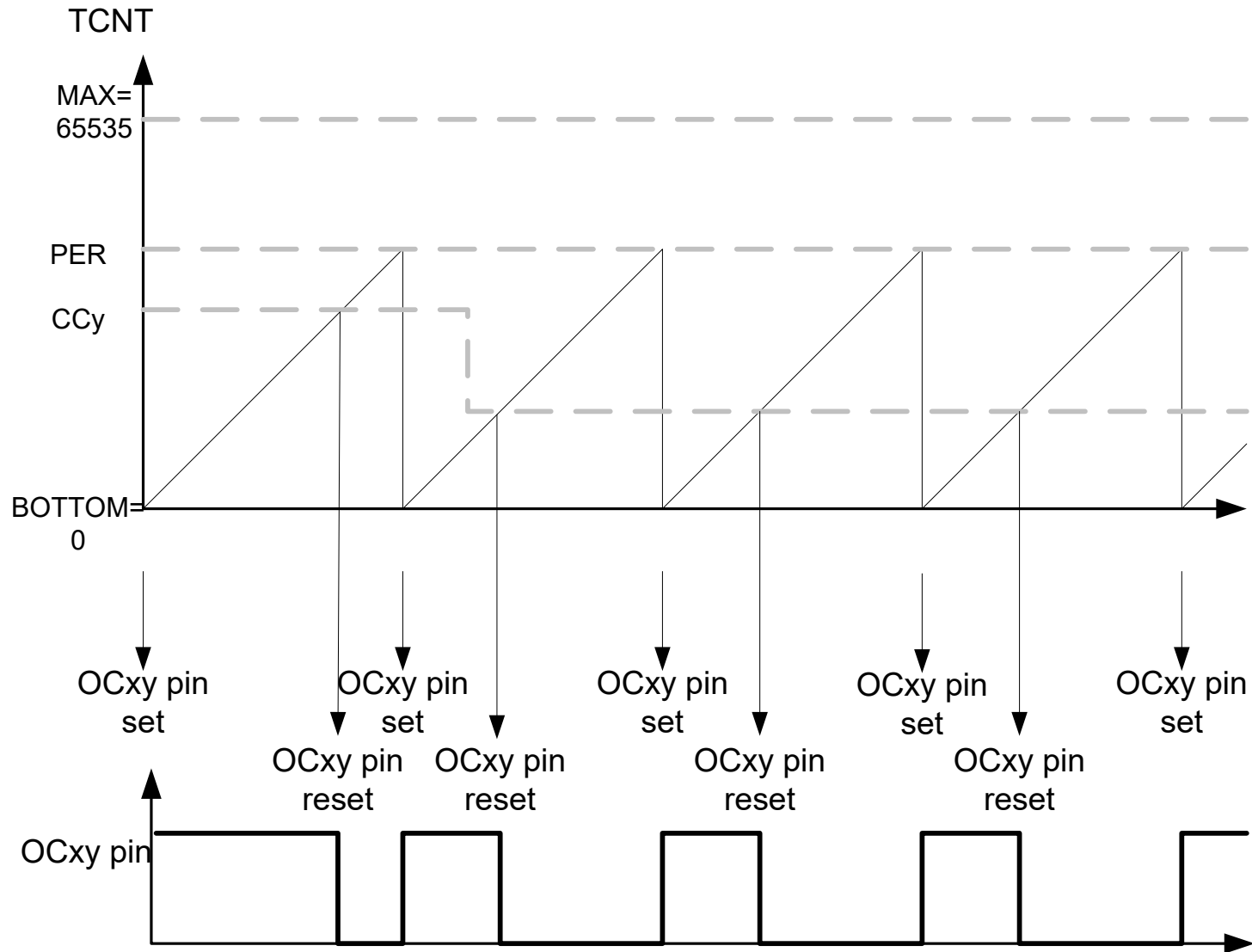


# FRQ WGM (1)



- Belangrijke voorwaarden:
  - OCxA pin staat als uitgang
  - FRQ WGM geselecteerd
  - CCA kanaal is aangeschakeld
  - Event action is uitgeschakeld voor de TC

# SS PWM WGM



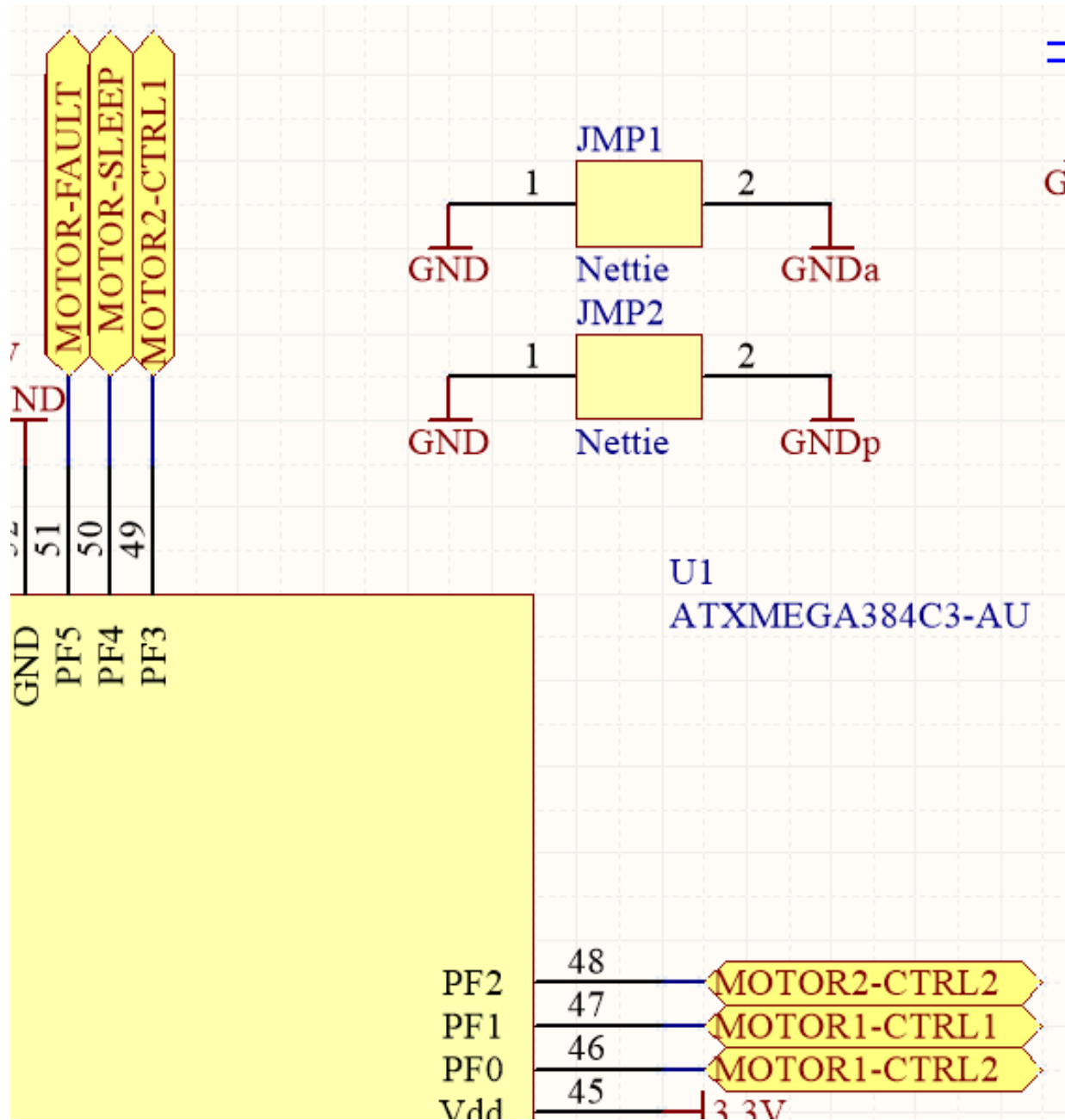
## SS PWM WGM (2)

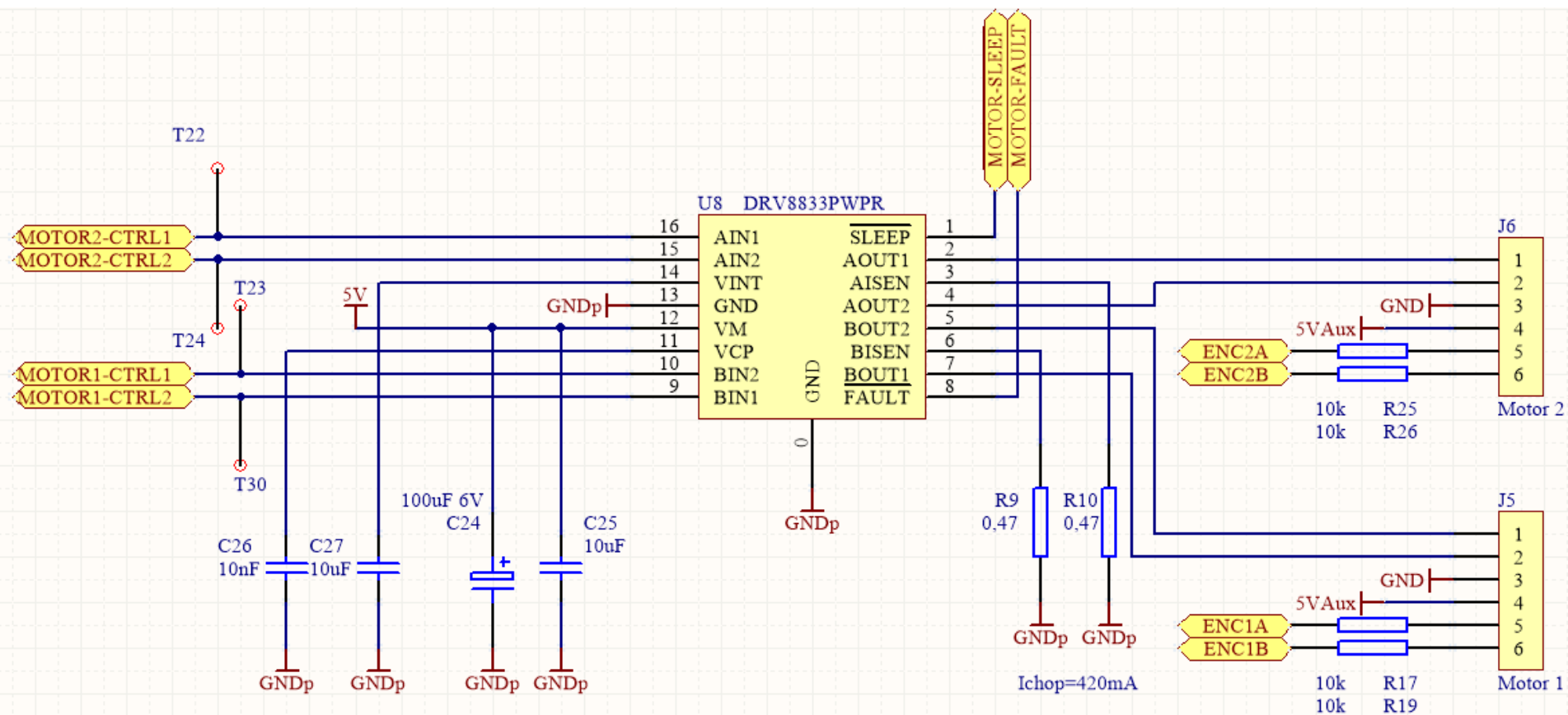
- Belangrijke voorwaarden:
  - OCxy pin staat als uitgang
  - SS PWM WGM geselecteerd
  - CCy kanaal is aangeschakeld
  - Event action is uitgeschakeld voor de TC





# Hardware: TC (Motor aansturing)

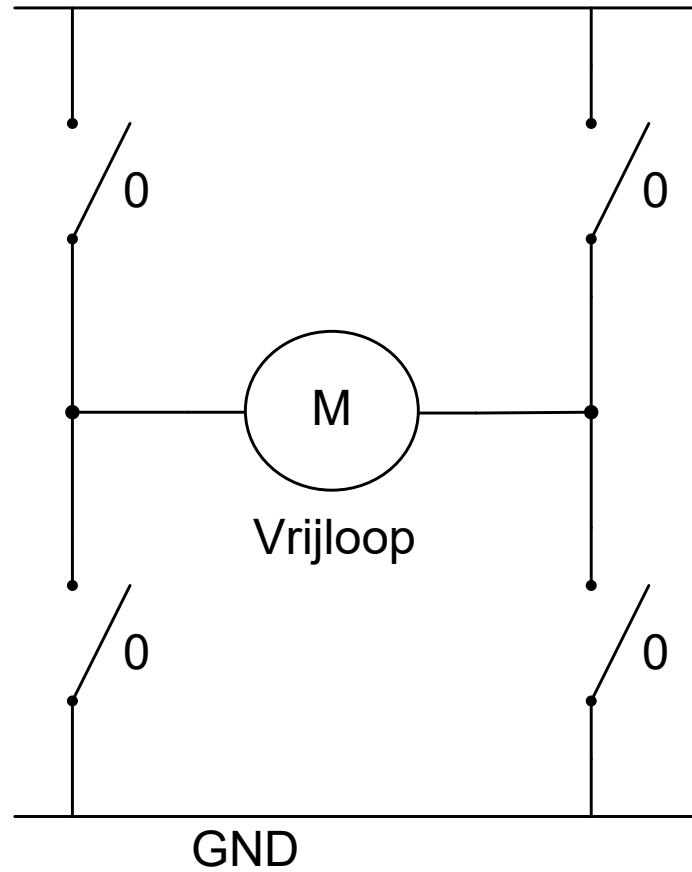




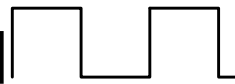
CTRL1=0

CTRL2=0

V<sub>CC</sub>

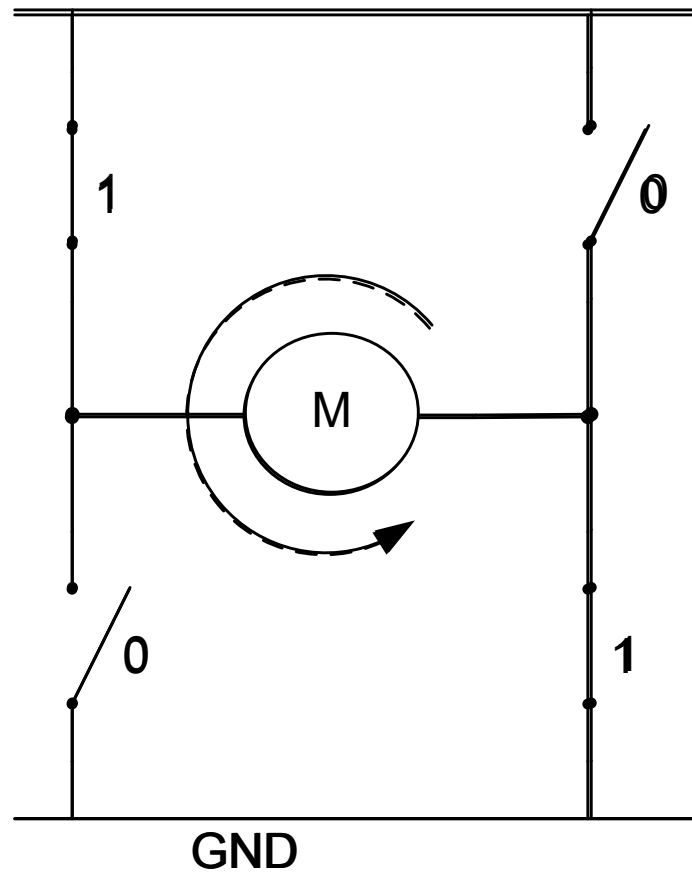


CTRL1=1



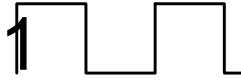
CTRL2=0

Vcc

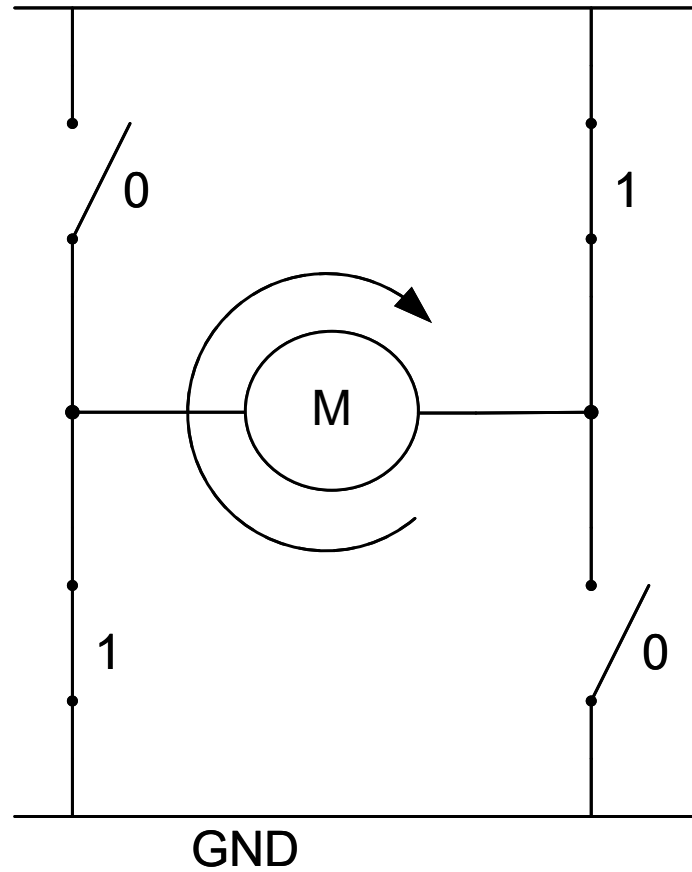


**CTRL1=0**

**CTRL2=1**

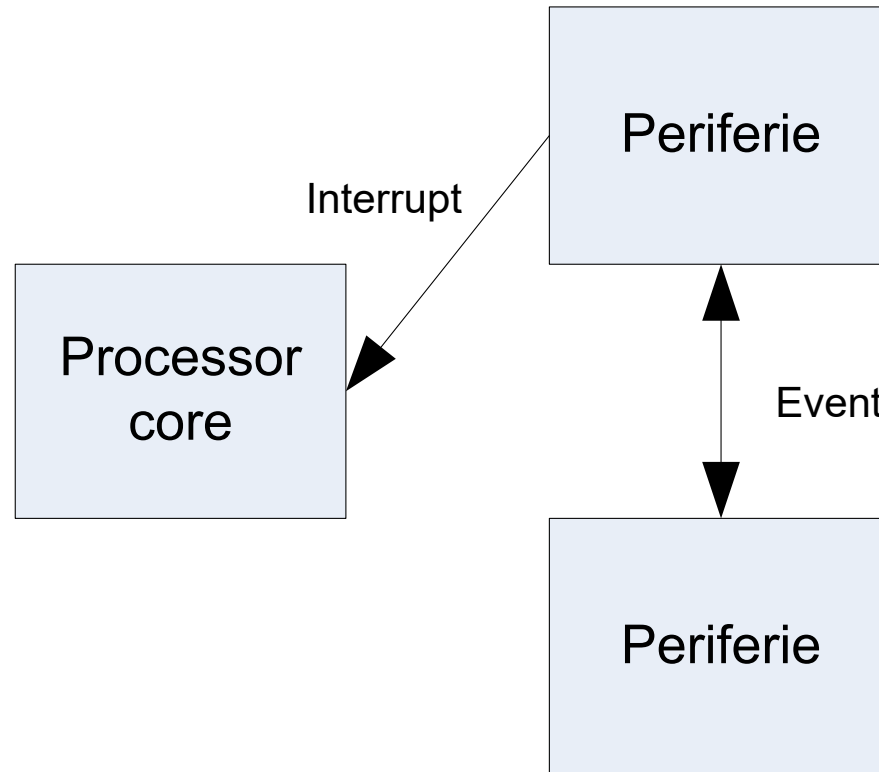


$V_{CC}$





# Signalering: interrupts en events



# Events

- Flexibel signaleringssysteem eigen aan ATXMEGA
- Signalering tussen periferie onderling
- Verbindingen zijn configureerbaar
- Vb: aflopen van een Timer laat automatisch een ADC conversie opstarten



# Interrupts

- 10-tallen bronnen (ADC, USART, flankdetectie GPIO, ...)
- Elke bron heeft een of meerdere individuele ISR's
- Bij optreden interrupt: ISR (Interrupt Service Routine) wordt uitgevoerd.
- Nadien hervat de originele programmaflow
- 4 niveau's in dalende prioriteit (PRI): NMI, High, Medium, Low
- Hogere prioriteits interrupt kan een lagere prioriteit ISR onderbreken





# Voorwaarden voor uitvoering INT X met PRI Y

- Interrupt X is enabled met PRI Y (zie registers interrupt bron, bv **USARTD0.CTRLA**)
  - prioriteit Y is enabled (**PMIC.CTRL** register)
  - GIE bit is 1 (**SREG** register)
  - GEEN ISR met  $PRI \geq PRI Y$  wordt momenteel uitgevoerd
  - Interrupt FLAG X is hoog
- 
- De ISR voor de betreffende interrupt bestaat!  
Indien niet → reset / onvoorspelbaar gedrag

# ISR's onder Atmel STUDIO

- Wordt aanzien als een speciale functie zonder parameters of return waarde
- `#include <avr/interrupt.h>` is nodig

• Voorbeeld:

## **21.17 Interrupt Vector Summary**

| Offset | Source   | Interrupt description                      |
|--------|----------|--------------------------------------------|
| 0x00   | RXC_vect | USART receive complete interrupt vector    |
| 0x02   | DRE_vect | USART data register empty interrupt vector |
| 0x04   | TXC_vect | USART transmit complete interrupt vector   |

- `USART0_RXC_vect`: ISR vector naam.
- Vector namen: zie einde van elke sectie in de proc. manual, bv p263



University of Antwerp

Faculty of Applied  
Engineering

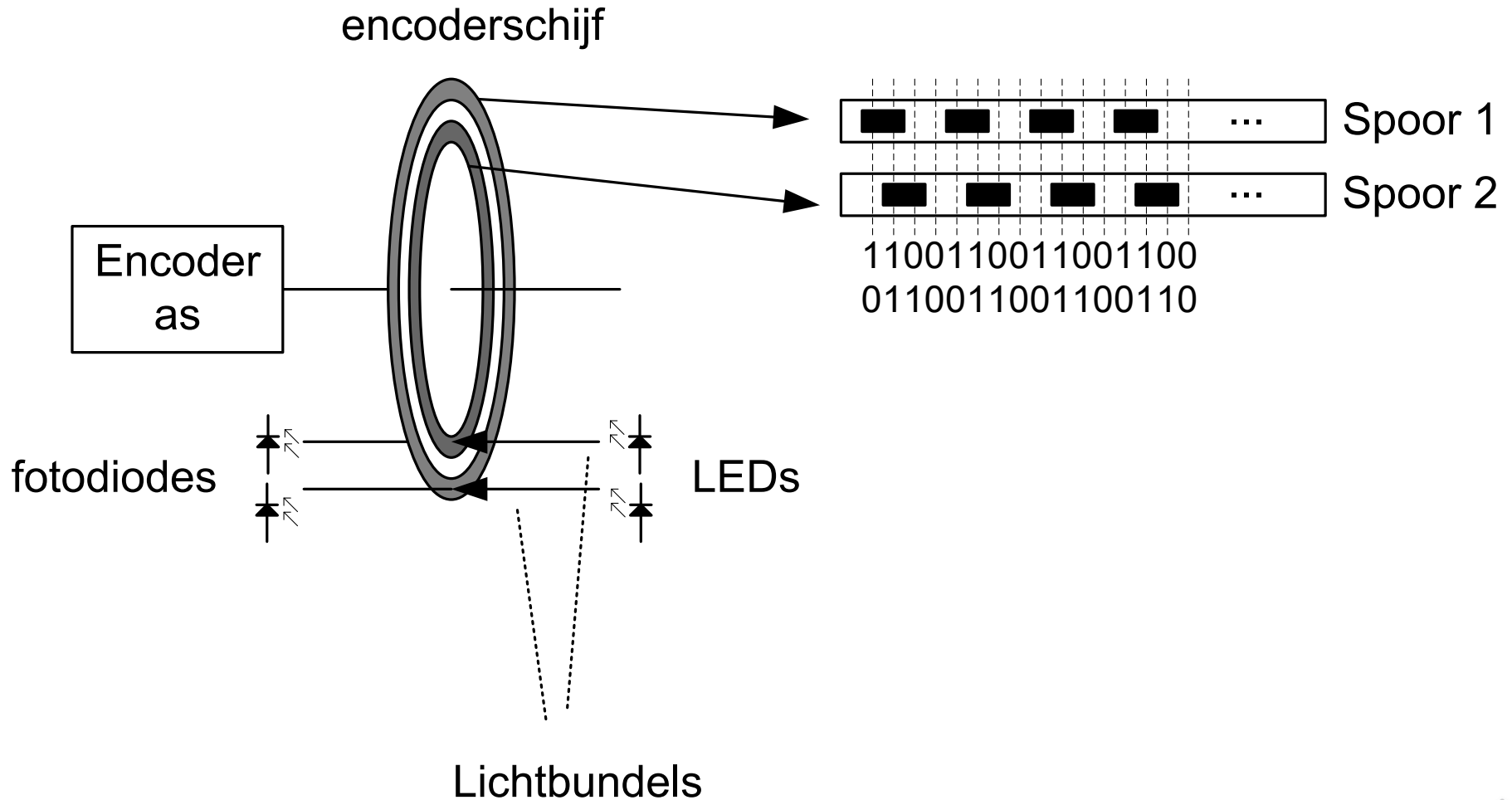
# GPIO interrupts

# GPIO interrupts

- Elke GPIO poort (A,B,C,...) heeft 2 configureerbare interrupts.
- Via `PORTx.INTCTRL` aan/uit schakelbaar
- Via `PORTx.INT0MASK` en `PORTx.INT1MASK` te koppelen aan een gewenste pin
- Via `PORTx.PINnCTRL` selecteerbaar op welke gebeurtenis de interrupt activeert (laag niveau / stijgende flank / dalende flank / willekeurige flank)
- Triggert de ISR met naam `PORTx_INT0_vect` of `PORTx_INT1_vect`
- ISR triggering onder invloed van `PORTx.INTFLAGS`, wordt gereset bij oproep ISR (p102)



# Optische encoder



# Incrementele encoder

- Afleidbaar uit encoder informatie:
  - Draaisnelheid (tijd tussen de code stappen)
  - Draairichting (positieve / negatieve code stap)
  - Relatieve positie: stijgende / dalende stappen tellen, bijhouden in telvariabele

|           |   |   |   |   |   |   |   |   |   |   |   |   |
|-----------|---|---|---|---|---|---|---|---|---|---|---|---|
| Spoor 1   | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 |
| Spoor 2   | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| Gray code | 1 | 2 | 3 | 0 | 1 | 2 | 3 | 0 | 1 | 2 | 3 | 0 |

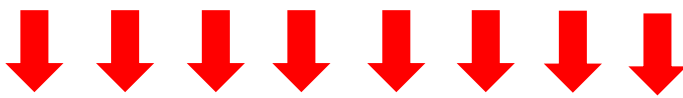


# Relatieve positie:implementatie

- We kijken naar alle mogelijke overgangen tussen de verschillende codes
- Gray code: slechts één bit verandert per overgang
- We bekijken een bit die verandert als een flank → interrupts

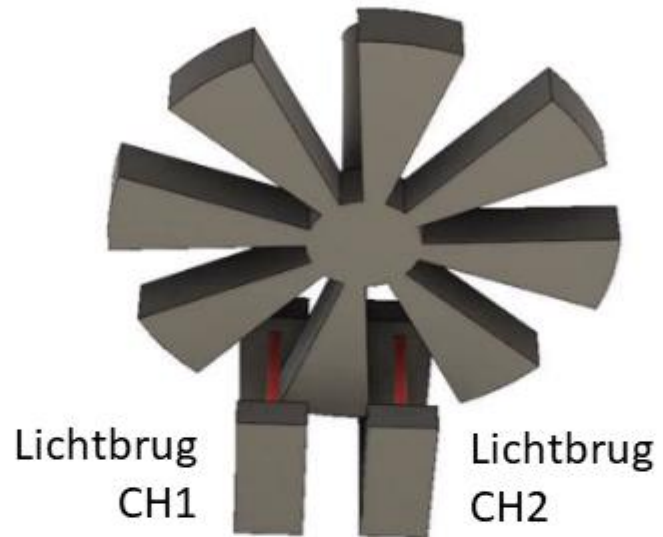


|           |   |   |   |   |   |   |   |   |   |   |   |   |
|-----------|---|---|---|---|---|---|---|---|---|---|---|---|
| Spoor 1   | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 |
| Spoor 2   | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| Gray code | 1 | 2 | 3 | 0 | 1 | 2 | 3 | 0 | 1 | 2 | 3 | 0 |



|         |   |   |   |   |   |   |   |   |
|---------|---|---|---|---|---|---|---|---|
| Spoor 1 | ↑ | ↓ | ↑ | ↓ | 0 | 0 | 1 | 1 |
| Spoor 2 | 0 | 0 | 1 | 1 | ↑ | ↓ | ↑ | ↓ |
| Stap    | + | - | - | + | - | + | + | - |





CH1 0

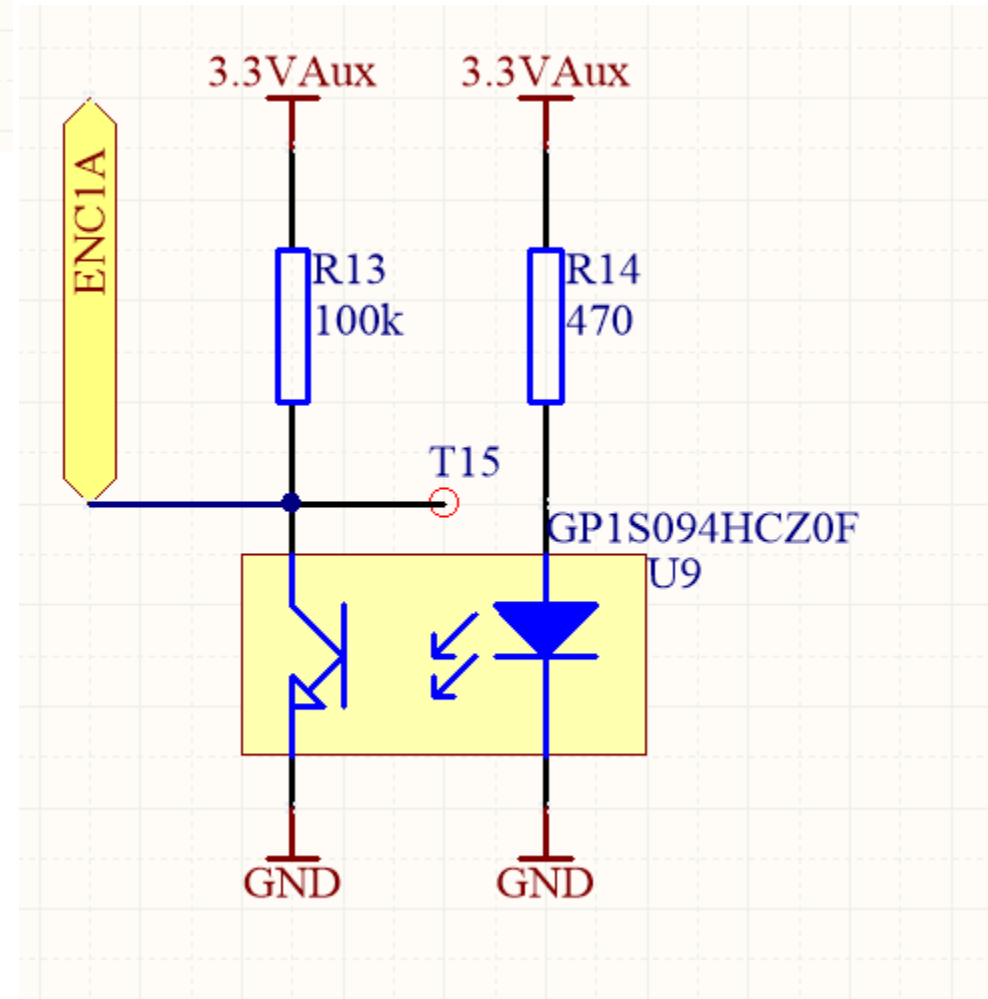
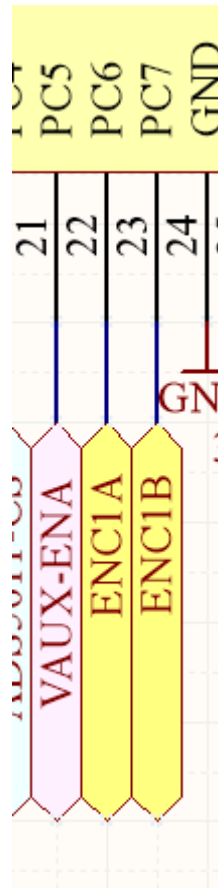
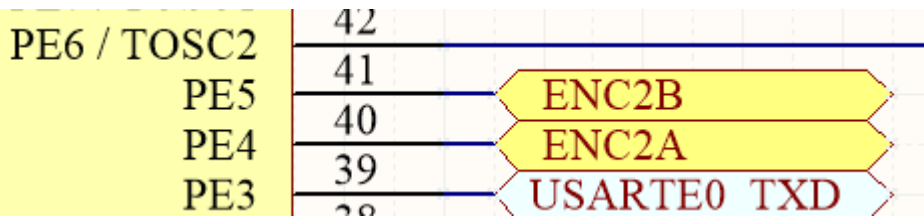
CH2 0

Gray code 0





# Hardware: IO (Incrementale encoder)



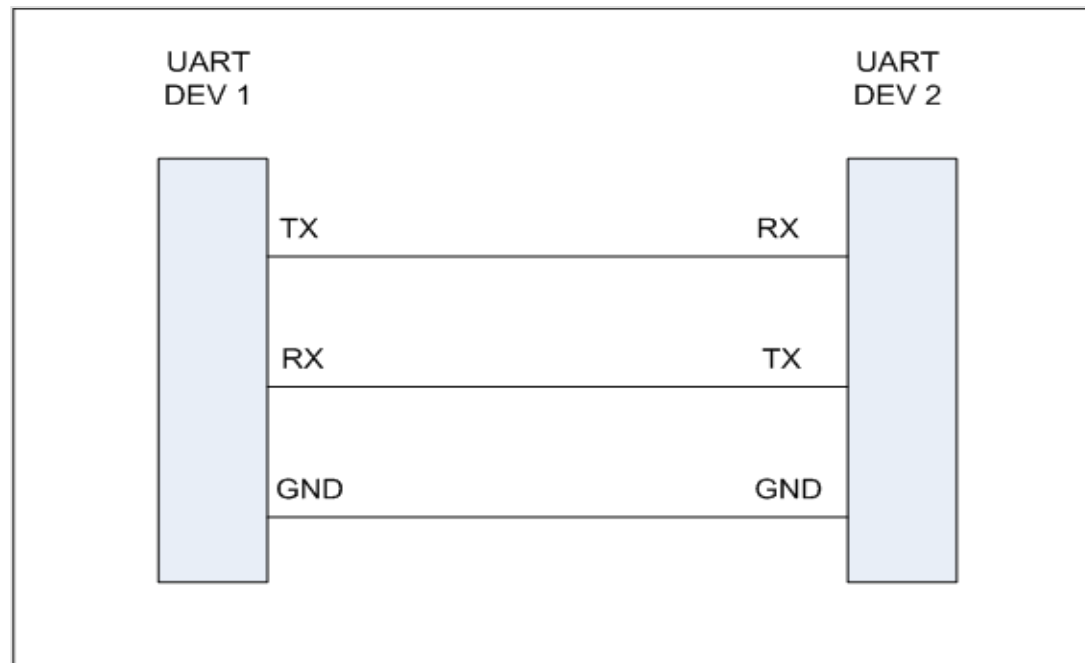


University of Antwerp  
Faculty of Applied  
Engineering

**USART**  
**(Universal Synchronous /**  
**Asynchronous Receiver and**  
**Transmitter)**  
**Manual p 244 e.v.**

# Algemeen

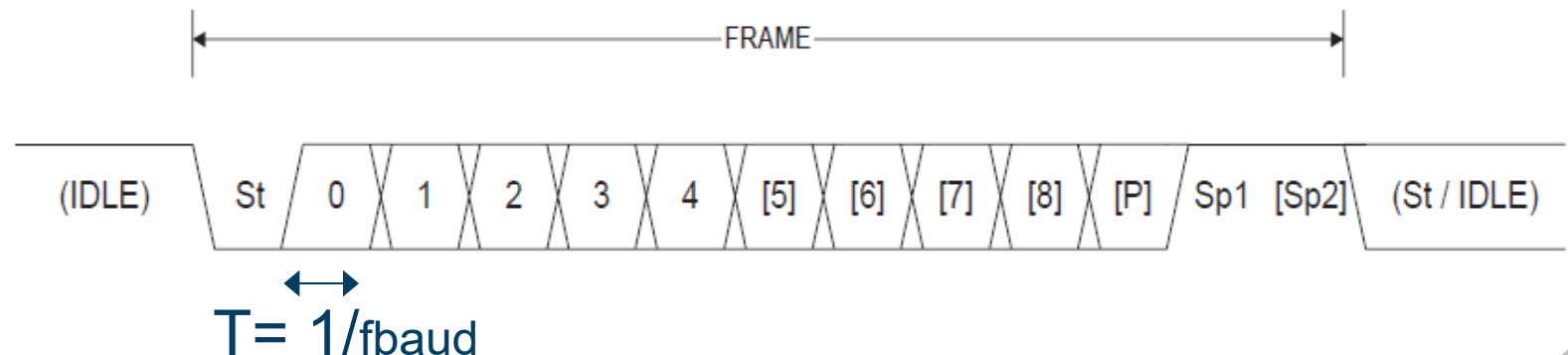
- Meestal als UART gebruikt → compatibel met RS-232
- Point to point seriële verbinding
- Geen ingebed klok signaal
- Tot 3Mbps
- Doel: debugging interface, management console, communicatie met GPS, andere  $\mu$ C, sommige sensoren, ...
- USARTC (PORTC), USARTD (PORTD), USARTE (PORTE)



# Karakter opbouw

Karakter:

- Rusttoestand: lijn hoog
- 1 startbit
- 5 – 9 databits (meestal 8). Eerst LSB
- Optioneel: pariteitsbit
- 1 of 2 stopbits (meestal 1)



# Bit timing (1)

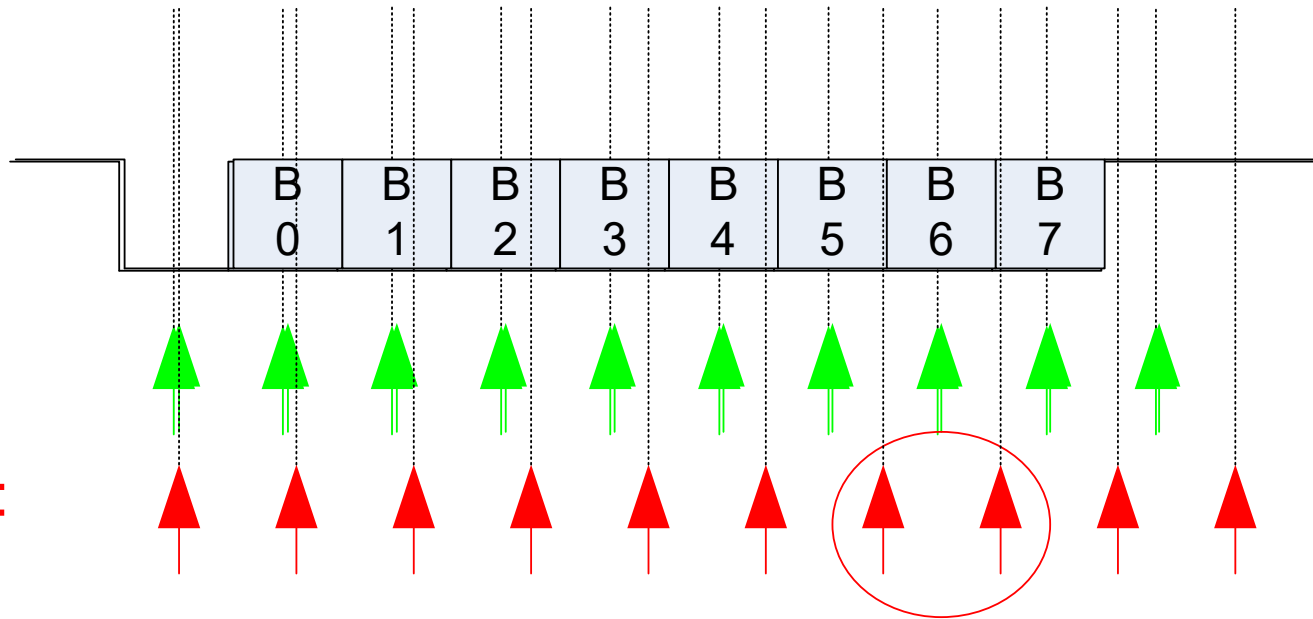
- Baudrate = bitrate =  $f_{\text{baud}}$  is instelbaar
- USART heeft een CLK ( $f_{\text{rx}}$ ) nodig.
- $f_{\text{rx}} = f_{\text{per}} / \text{DIV}$
- $f_{\text{baud}} = f_{\text{rx}} / 16$  (of /8 in CLK2X mode)
- $f_{\text{baud}}$  is afgeleid van  $f_{\text{per}}$  (32 MHz) via een programmeerbare deler (DIV)
  - De deler moet zo correct mogelijk ingesteld worden.  $f_{\text{baud zender}}$  t.o.v.  $f_{\text{baud ontvanger}}$  : max 4,5% afwijking bij 8 databits (zie p253 in 'proc manual')

## Bit timing (2)

**TX:**

**RX OK:**

**RX ERR:**



# Fractionele deler (1)

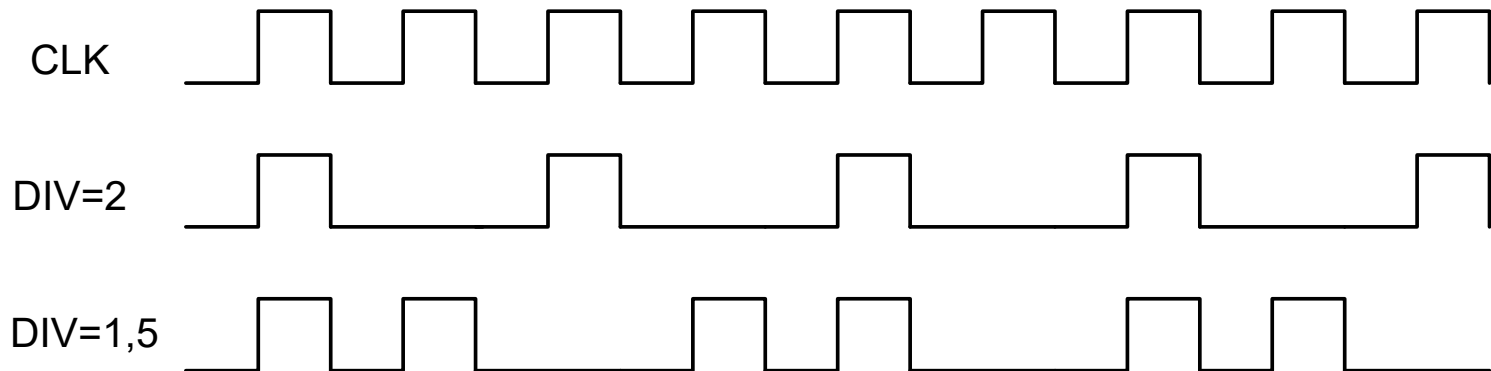
- **Probleem:**

- Stel  $f_{\text{per}} = 16 \text{ MHz}$
- Deelfactor  $\text{DIV}=833 \rightarrow f_{\text{baud}} = 16 \text{ MHz} / 16 / 833 = 1200,4 \text{ baud}$
- Deelfactor  $\text{DIV}=834 \rightarrow f_{\text{baud}} = 16 \text{ MHz} / 16 / 834 = 1199,0 \text{ baud}$
- Deelfactor  $\text{DIV}=1 \rightarrow f_{\text{baud}} = 16 \text{ MHz} / 16 / 1 = 1000 \text{ kbaud}$
- Deelfactor  $\text{DIV}=2 \rightarrow f_{\text{baud}} = 16 \text{ MHz} / 16 / 2 = 500 \text{ kbaud}$

→ Bij lage DIV (hoge baudrate) is de baudrate grof instelbaar

# Fractionele deler (2)

- Oplossing: fractionele deler
- Deler met  $DIV = \text{rationeel getal}$
- Bijvoorbeeld:  $DIV=1,5 \rightarrow 2 \text{ op } 3 \text{ CLK cycli doorlaten}$





## Fractionele deler (3)

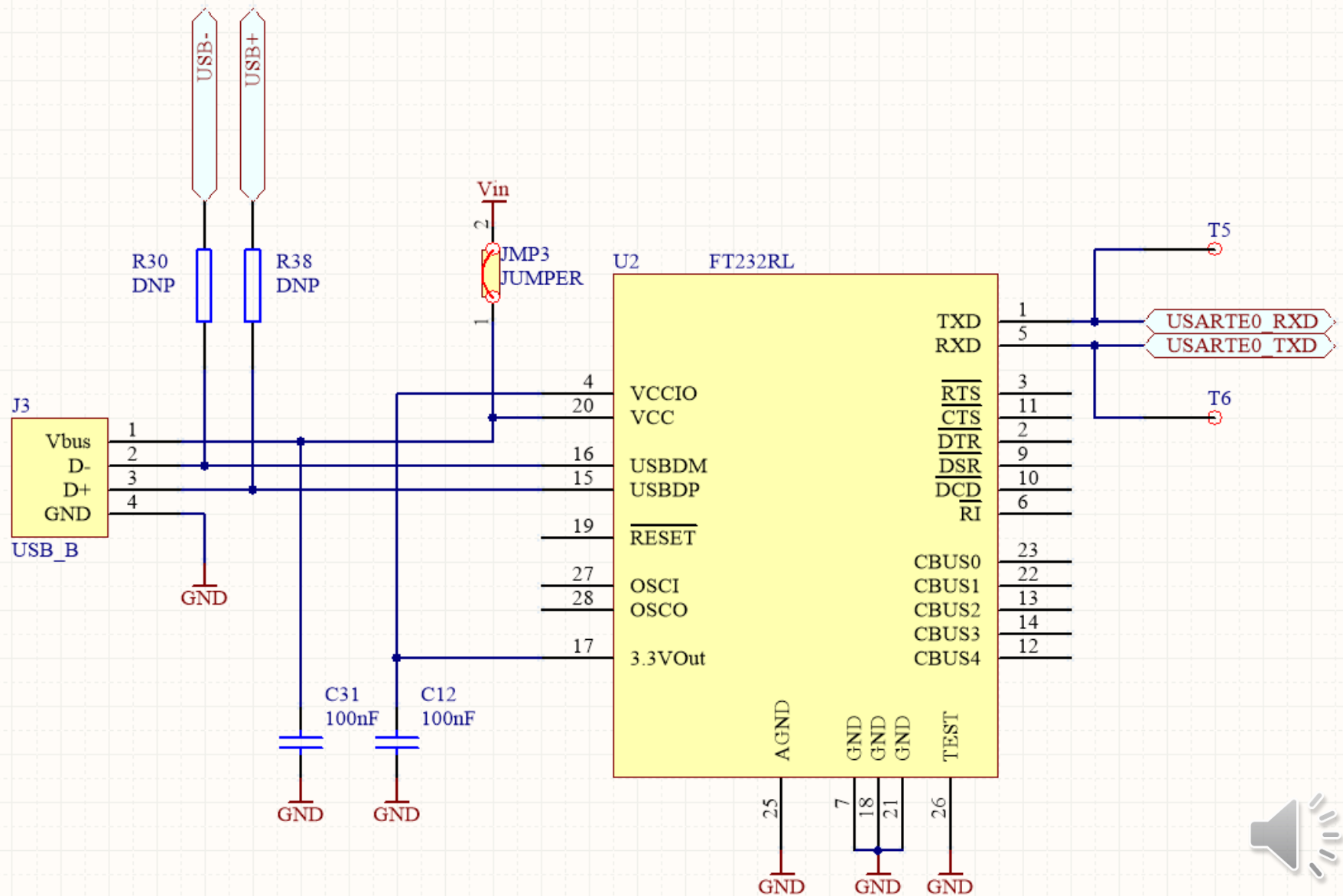
- DIV: opgesplitst in BSCALE en BSEL (zie proc manual p246)
- BSCALE: [-7,+7] BSEL: [0,4095]

$$BSCALE < 0 : f_{\text{baud}} = \frac{f_{\text{PER}}}{16((2^{BSCALE} \cdot BSEL) + 1)}$$

$$BSCALE \geq 0 : f_{\text{baud}} = \frac{f_{\text{PER}}}{2^{BSCALE} \cdot 16(BSEL + 1)}$$



# Hardware: USART (terminal poort)





University of Antwerp  
Faculty of Applied  
Engineering

# **ADC (*Analog to Digital Converter*)**

## **Manual p 279 e.v.**



- Resolutie = 8 bit of 12 bit
- Enkelpolige meting (t.o.v. GND) of differentiële meting (2 aansluitingen)
- Resultaat: signed of unsigned (instelbaar)
- Referentiespanning: intern of extern
- In differentiële mode: optioneel analoge versterker/verzwakker:  $/2$ ,  $\times 2$ ,  $\times 4$ ,  $\times 8$ ,  $\times 16$ ,  $\times 32$ ,  $\times 64$
- Interrupt mogelijk op voltooiing van conversie



# Timing

- Heeft klok nodig, afgeleid van periferieklok
- $f_{\text{ADCmax}} = 1,8\text{MHz}$
- Totale metingstijd:

$$t_{\text{total}} = t_{\text{sample}} + t_{\text{conv}}$$

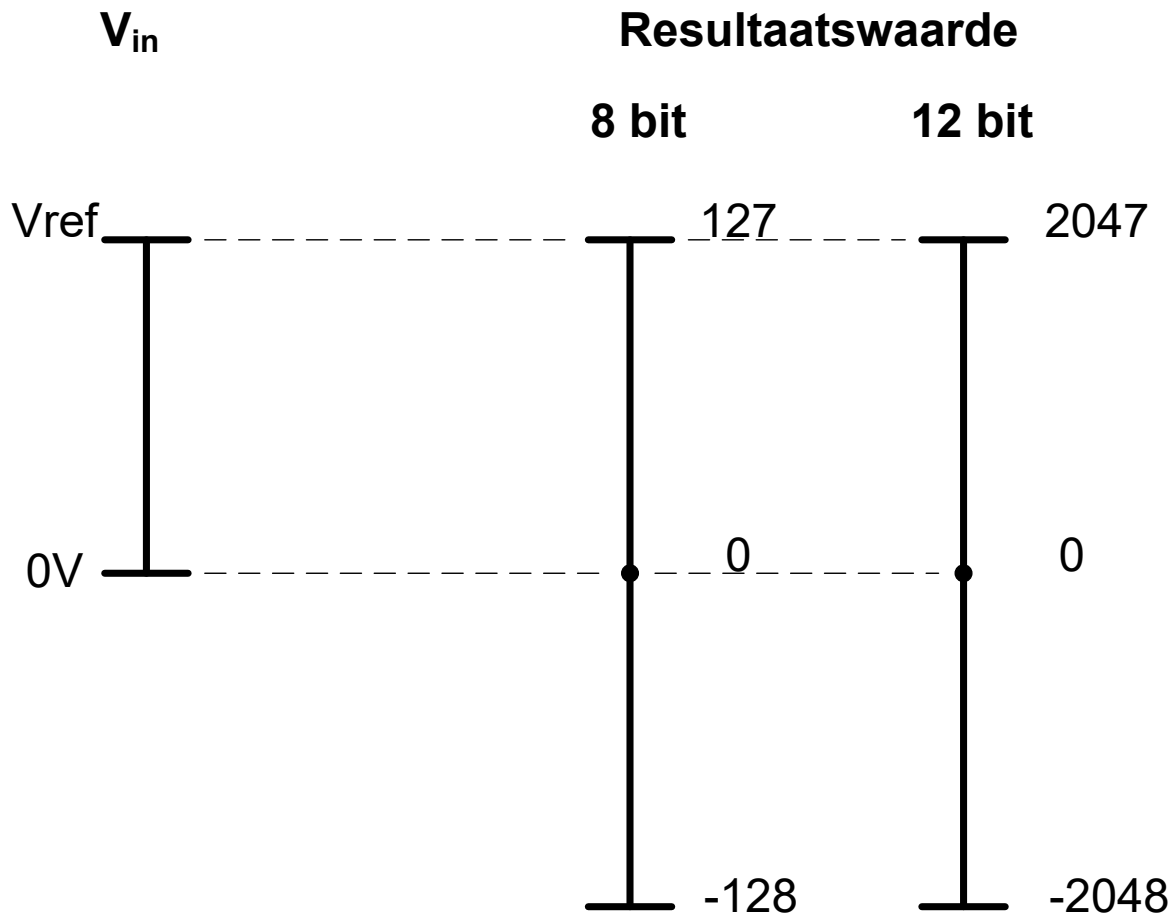
- Instelbare sample tijd:  $0,5 T_{\text{ADC}}$  tot  $32 T_{\text{ADC}}$
- Hogere sample tijd  $\rightarrow$  hogere ingangsimpedantie
- Conversie tijd:  $5,5 T_{\text{ADC}}$  tot  $10 T_{\text{ADC}}$
- Conversie tijd is afhankelijk van resolutie, gain

$\rightarrow$  max ADC meetfrequentie: ca 300kHz



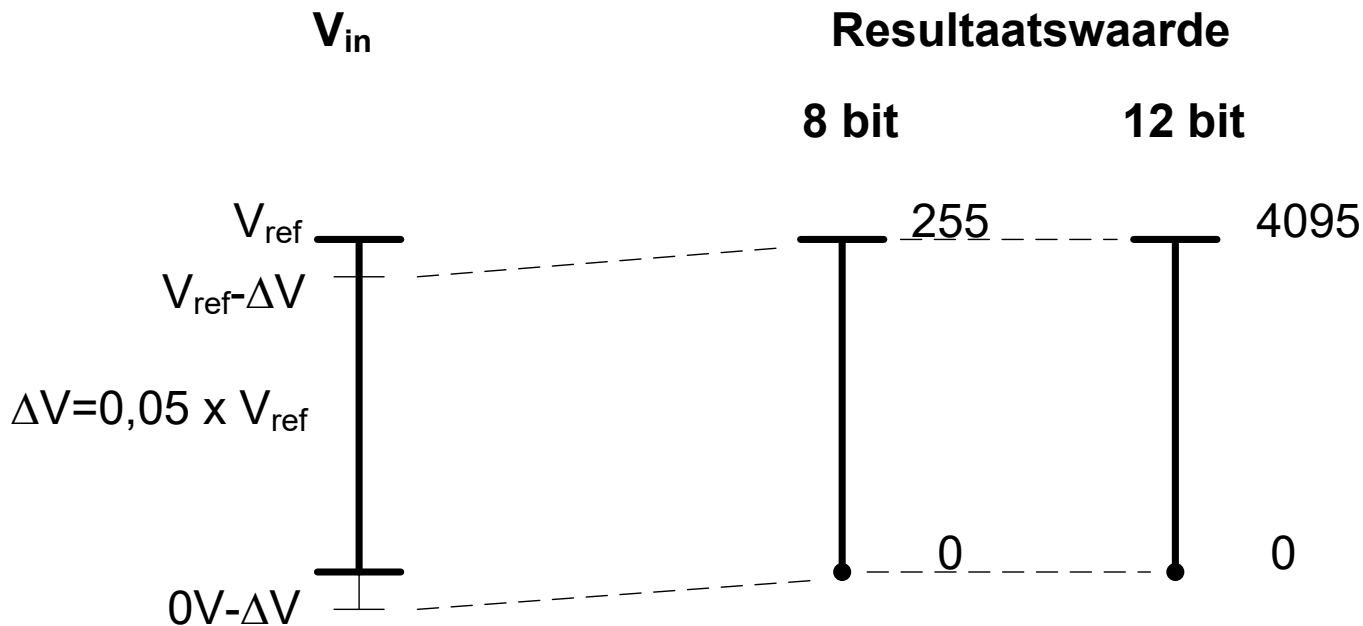
# ADC resultaatswaarde(1)

## Enkelpolig, signed



# ADC resultaatswaarde(2)

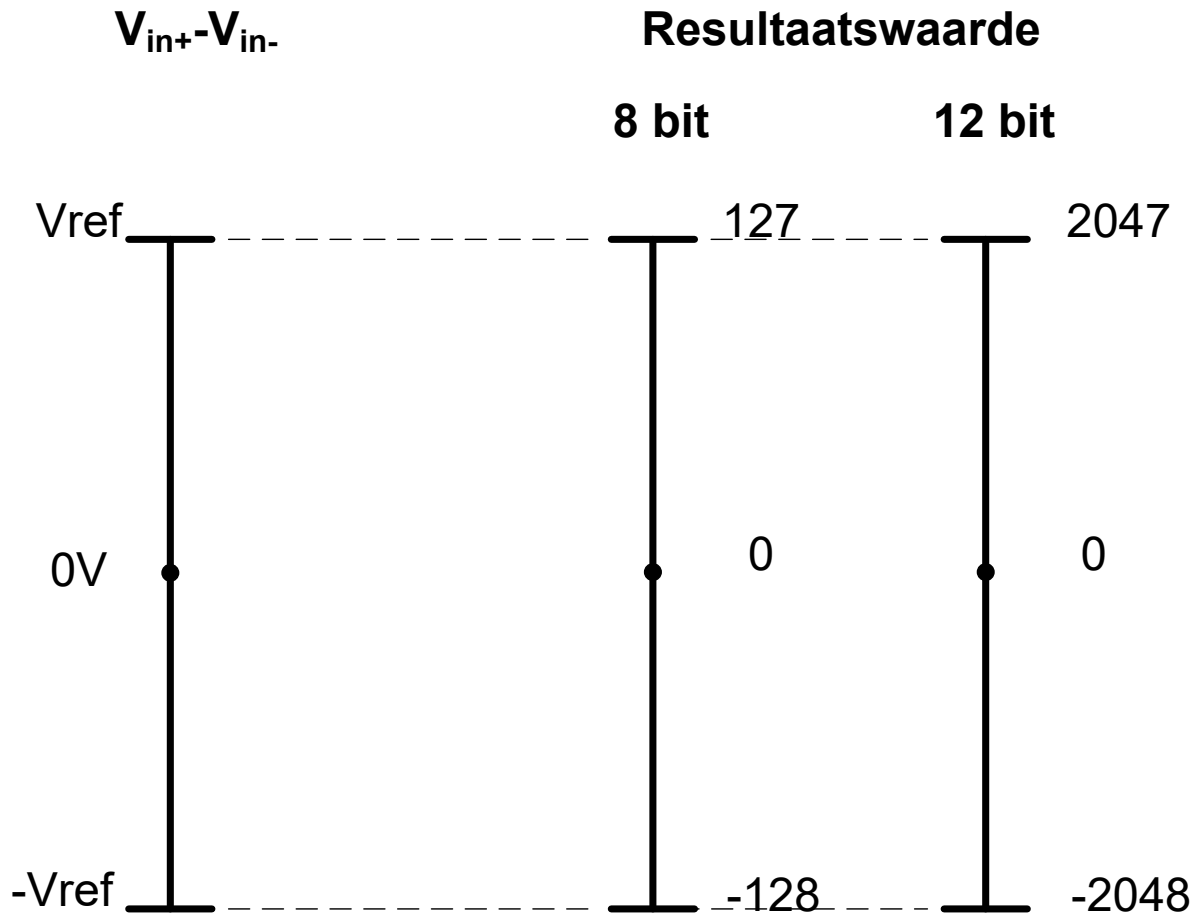
## Enkelpolig, unsigned



- Meetbereik is 5% negatief verschoven om eventuele offset fouten van de ADC te compenseren

# ADC resultaatswaarde(3)

## Differentieel, signed





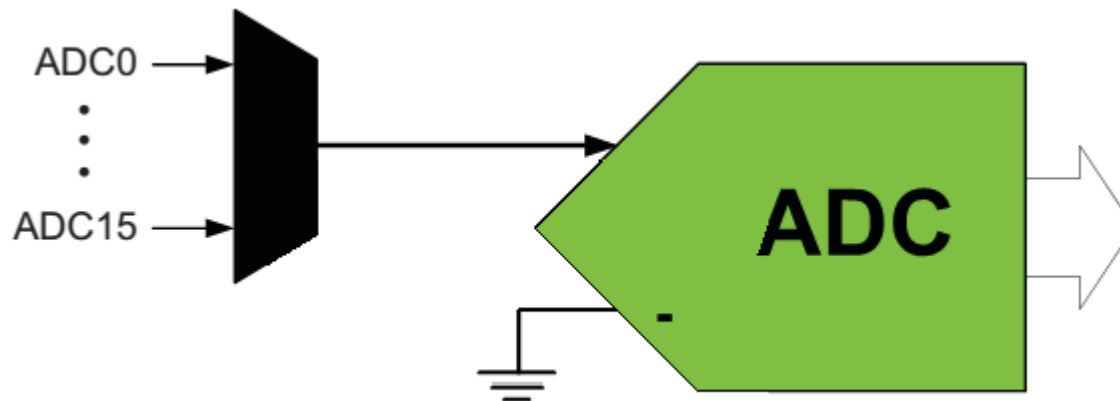
# ADC registers (1)

- **ADCA.CTRLA, ADCA.CTRLB, ADCA.CH0.CTRL:** configuratie, opstart van een conversie
- **ADCA.REFCTRL:** configuratie van referentie spanning
- **ADCA.SAMPCTRL:** configuratie van sample tijd
- **ADCA.PRESCALER:** configuratie van klok frequentie
- **ADCA.INTFLAGS:** interrupt vlag register. Ook nodig om te pollen wanneer conversie voltooid is.
- **ADCA.CH0.RES:** ADC data register (conversieresultaat)

## ADC registers (2)

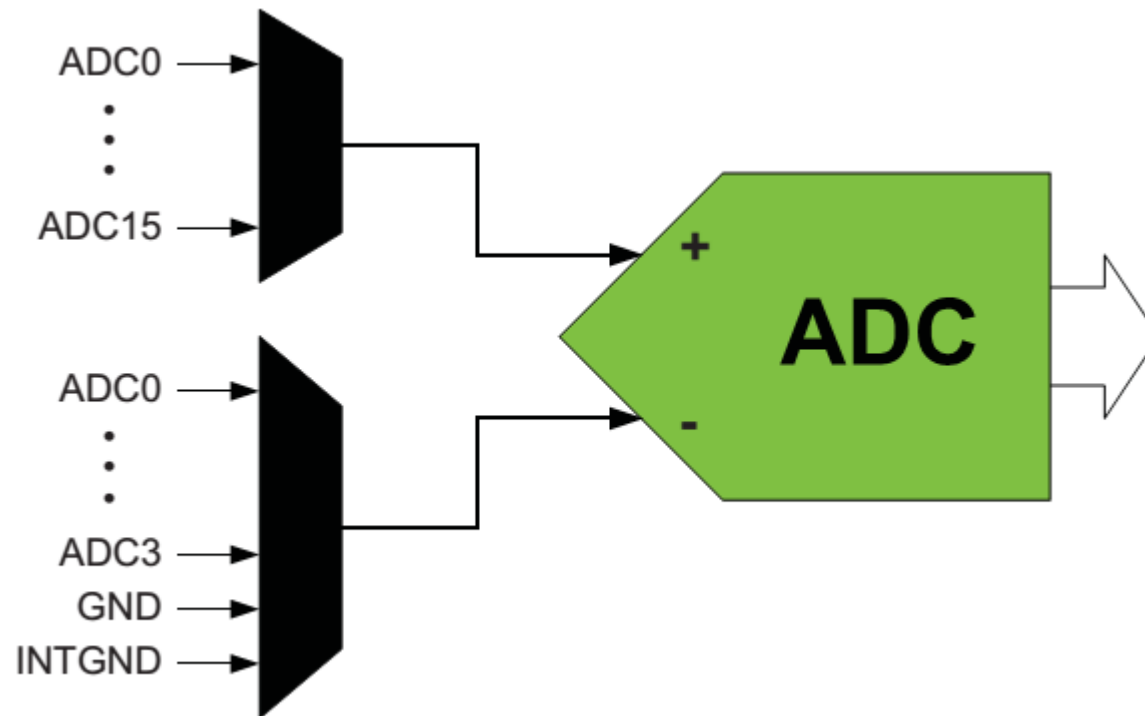
- **ADCA.CH0.MUXCTRL:** Slechts één ADC, wel gemultiplext. Selectie van te meten pinnen.

### Single ended mode



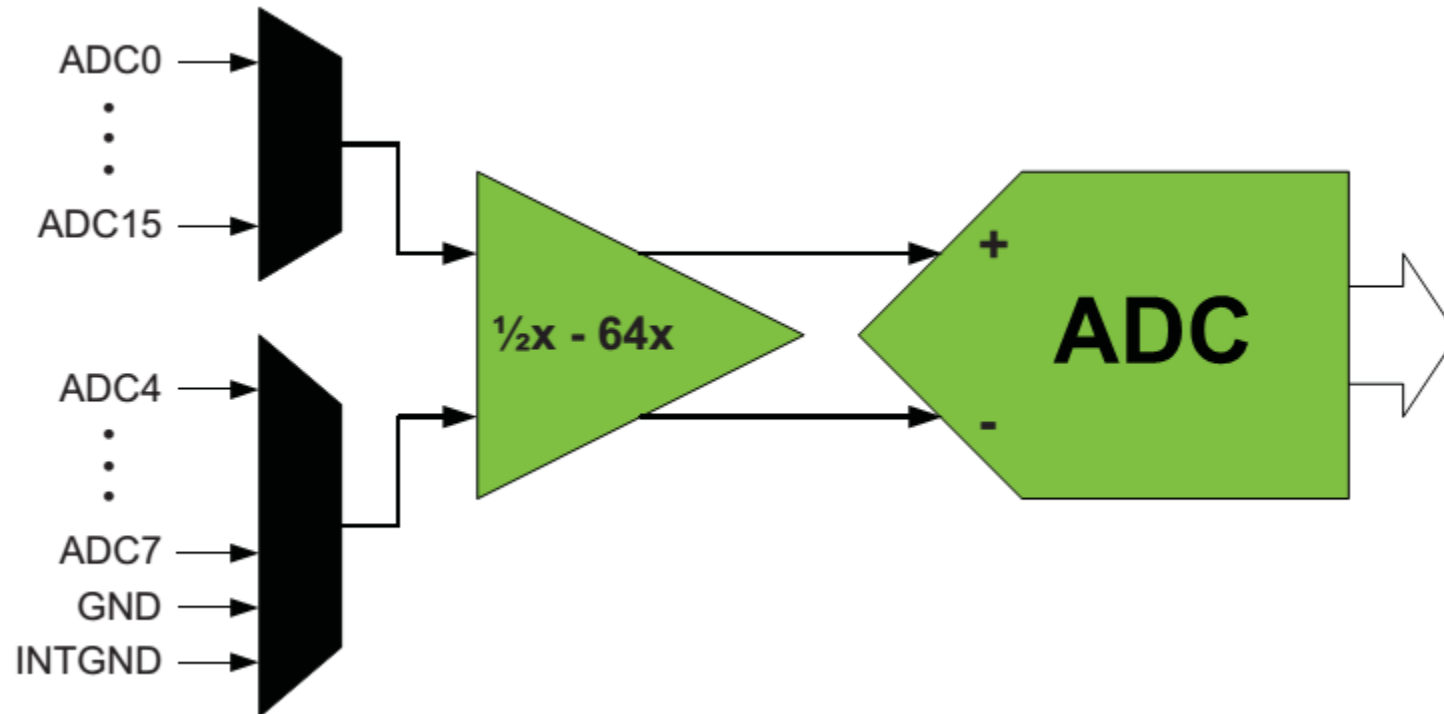
# ADC registers (3)

## Differential mode without gain

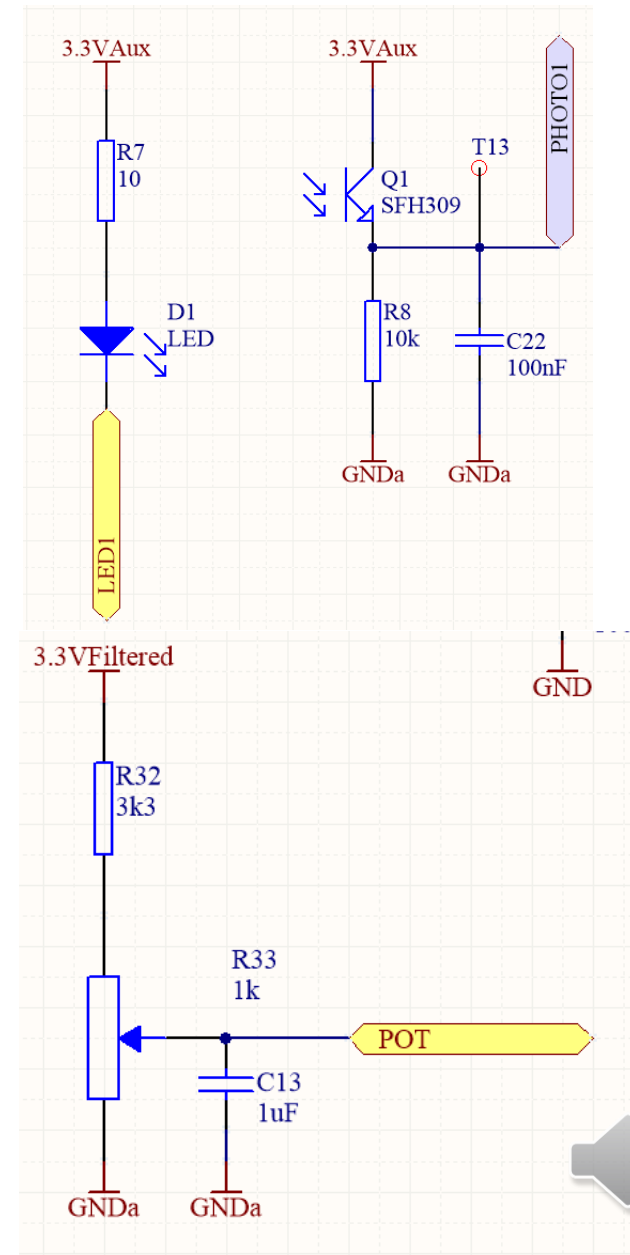
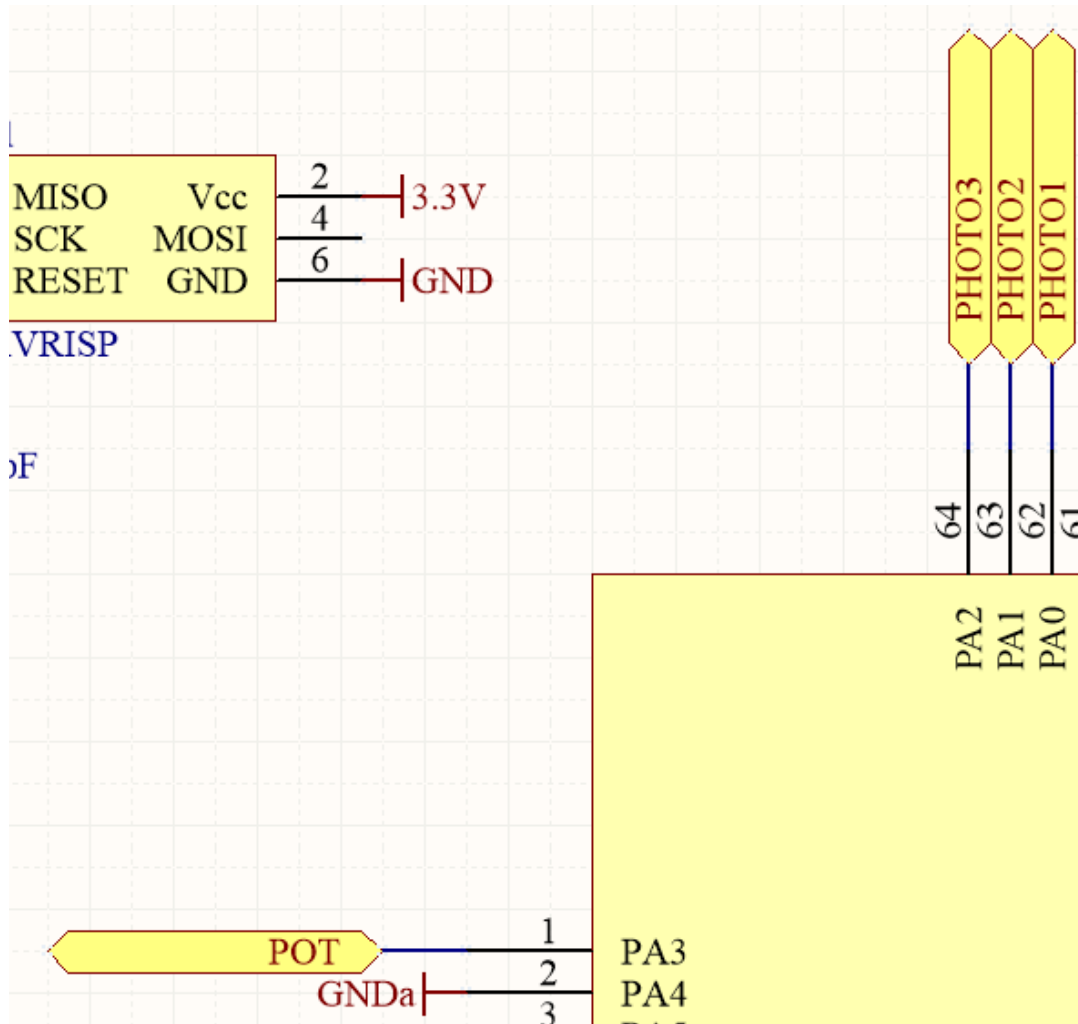


# ADC registers (4)

## Differential mode with gain



# Hardware: ADC (lijnvolger, potmeter)





University of Antwerp  
Faculty of Applied  
Engineering

# Power management, RTC

# Power management

- Selectief uitschakelen van peripherals
- Verlagen van klokfrequentie
- Uitschakelen van klokdomeinen

→ Energie sparen door klok uit te schakelen is gemakkelijk.

Moeilijker: zorgen dat de CPU terug wakker wordt!

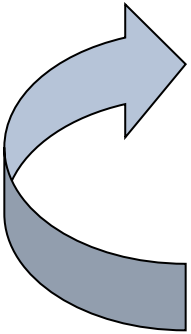
**Table 7-1.** Active clock domains and wake-up sources in the different sleep modes.

| Sleep Modes      | Active Clock Domain |           | Oscillators         |                  | Wake-up Sources              |                              |                            |                |
|------------------|---------------------|-----------|---------------------|------------------|------------------------------|------------------------------|----------------------------|----------------|
|                  | CPU Clock           | RTC Clock | System Clock Source | RTC Clock Source | Asynchronous Port Interrupts | TWI Address Match Interrupts | Real Time Clock Interrupts | All Interrupts |
| Idle             |                     | X         | X                   | X                | X                            | X                            | X                          | X              |
| Power down       |                     |           |                     |                  | X                            | X                            |                            |                |
| Power save       |                     | X         |                     | X                | X                            | X                            | X                          |                |
| Standby          |                     |           | X                   |                  | X                            | X                            |                            |                |
| Extended standby |                     | X         | X                   | X                | X                            | X                            | X                          |                |



- Universele strategie:

- Systeem x ms laten slapen (power save)
- Wakeup door Timer (RTC)
- Check of er iets moet gedaan worden



# Power management registers

- SLEEP.CTRL: instelling van sleep mode
- SLEEP.PRGEN, SLEEP.PRPA, SLEEP.PRPC, SLEEP.PRPD, SLEEP.PRPE, SLEEP.PRPF: aan/uitschakelen van peripherals
- sleep\_cpu() : voert 'sleep' instructie uit om CPU in sleep mode te zetten.

# RTC (real time counter)

- Low power TC met eigen klok
  - Interne kristal osc.
  - Extern signaal
  - Interne RC osc (gecalibreerd)
  - Interne RC osc (very low power)
- Kan processor uit sleep halen en/of interrupt genereren

→ Ideale bron om periodisch CPU uit sleep te halen!

# RTC registers

- **CLK.RTCCTRL**: klok instellingen van RTC
- **RTC.CTRL**: RTC prescaler instellingen
- **RTC.INTCTRL**: interrupt instellingen van RTC (compare of overflow INT)
- **RTC.PER**: periode register (idem TC)
- **RTC.COMP**: zoals TCx.CC register